Frame-Relative Counterfactual Causes for Explaining Synthesized Temporal Behavior (with an automata-theoretic trace-level test and a strategy-level reverse parity-game lift)

Abstract

Linear Temporal Logic (LTL) synthesis usually moves from a temporal specification to an automaton, then to a parity game, then to a winning strategy, and finally to a finite-state controller such as a Mealy machine. This forward transformation is intentionally many-to-one: different LTL specifications can yield the same controller, and different representations of winning strategies can describe the same behavior. Therefore, a controller by itself does not determine a unique original specification or a unique temporal reason for its behavior.

This document describes a reverse use of parity-game structure. The goal is not to reconstruct the original LTL formula and not to synthesize a new controller. Instead, given a Mealy machine, an optional original LTL specification, a reverse-causal frame, a temporal effect to explain, and an observed execution, the goal is to identify a minimal temporal cause. The input is best viewed as $(M, \varphi, \mathfrak{F}, E, \tau)$, where the Mealy machine M already supplies the input/output partition, φ remains ordinary LTL specification information, and $\mathfrak{F}$ is the causal frame used by the reverse procedure. The central question is: among the non-vacuous temporal conditions whose removal prevents the chosen effect from occurring on the closest admissible counterfactual executions, which is the most preferred (by default the smallest, most readable) under an explicitly chosen preference order? The resulting construction treats the Mealy machine as a fixed strategy, builds an automata-theoretic arena around the effect selected by the frame, and tests and synthesizes candidate explanations by counterfactual reasoning over that arena. The trace-level necessity test is, in every regime, a one-player shortest-edit search plus non-emptiness check (the frozen machine has no free move); emptiness alone decides only non-vacuity, while the counterfactual-dependence clause requires optimizing to the closest removal and then evaluating the effect. A genuinely two-player *reverse parity game* appears for a distinct, strategy-level notion — can the controller drop a commitment and still *guarantee* the effect against all environments? — which the frame reaches by re-opening the system's strategic choices. This strategy-level notion additionally requires a synthesis artifact — the game G_φ, the frozen strategy σ, and a declared editable vertex set $V_{\mathfrak{F}}$ — which M alone does not determine; it is not a function of (M, τ, E). We therefore describe the method as automata-theoretic counterfactual extraction, with a reverse parity-game generalization for strategy-level causes, and are careful throughout to keep the trace-level (one-player) and strategy-level (two-player) notions distinct.

1 Motivation

The standard synthesis pipeline can be summarized as

$$\varphi_{\mathrm{LTL}} \longrightarrow A_\varphi \longrightarrow G_\varphi \longrightarrow \sigma \longrightarrow M.$$

Here φ_{LTL} is an LTL specification, A_φ is an automaton recognizing the traces that satisfy the specification, G_φ is a parity game encoding the interaction between the environment and the system, σ is a winning strategy for the system, and M is a finite-state implementation of that strategy.

This forward pipeline loses information. A Mealy machine records how a controller reacts to inputs, but it does not record which LTL formula motivated that behavior. Many specifications may be realized by the same machine. Also, many different winning strategies may have implementations that are behaviorally equivalent. Consequently, the reverse problem

$$M \longrightarrow \varphi_{\mathrm{LTL}}$$

is not well-defined unless additional information is supplied.

The more useful reverse question is explanatory rather than reconstructive. We assume that we are given

$$(M, \varphi, \mathfrak{F}, E, \tau),$$

where $M = (S, s_0, I, O, \delta, \lambda)$ is the Mealy machine, which already fixes the input/output partition with $I \cap O = \emptyset$; $\varphi \in \mathrm{LTL}(I \cup O)$ is optional original specification information; $\mathfrak{F}$ is the reverse-causal frame; E is the temporal effect to be explained, a single (possibly conjunctive) ω-regular trace property drawn from a space $\mathcal{E}$ of candidate effects; and τ is the actual observed execution, the trace. The task is to compute or check a temporal property as a candidate cause, determine whether it belongs to the valid-cause set, and then select a minimal cause. Appendix B explains the relation between φ and $\mathfrak{F}$, and Appendix C explains the possible choices of E, including the TempoBench-compatible restriction.

To separate validity from minimality, let $\mathcal{K}_{\mathfrak{F}}$ be the space of candidate temporal explanations allowed by the causal frame. Also let $E_{\mathfrak{F}}$ be the effect selected by $\mathfrak{F}$; if no scoping is used, then $E_{\mathfrak{F}} = E$. The set

$$\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}}) \subseteq \mathcal{K}_{\mathfrak{F}}$$

contains all valid causes of $E_{\mathfrak{F}}$. An element

$$C_{\mathrm{valid}} \in \mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$$

is a valid cause when:

1. C_{valid} holds on the observed execution.
2. $E_{\mathfrak{F}}$ holds on the observed execution.
3. **Non-vacuity.** At least one admissible counterfactual execution violates C_{valid}; that is, removing the candidate is actually realizable under the frame.
4. If C_{valid} is removed in the closest admissible counterfactual executions, then $E_{\mathfrak{F}}$ no longer holds.

Condition 3 is essential: without it, any property that no admissible counterfactual can violate (for example a tautology, or any invariant that all admissible traces already satisfy) would pass condition 4 vacuously.

It is worth stating once, plainly, which notion of "cause" conditions 1–4 formalize. They encode *contingent counterfactual necessity*: C holds on the actual execution (1–2), removing C is realizable under the frame (3), and on the closest admissible executions that remove C the effect fails (4). This is a *counterfactual-necessity* reading of causation, not a sufficiency criterion and not Halpern–Pearl actual causation, which would add a minimal-sufficient-set requirement. We commit to this

necessity reading throughout, and the preference order below is the only place where additional selection (primarily preferring smaller, readable causes; see Section 8) enters. To avoid an overloaded vocabulary we reserve "logically weaker" for a larger language $\mathcal{L}(C)$ and "insufficient" for a candidate that, on its own, does not force the effect; these are different notions and we keep them lexically distinct. We avoid the unqualified phrase "weakest necessary condition": in standard logic the *strongest* necessary condition for $E_{\mathfrak{F}}$ (equivalently the weakest *sufficient* condition) is $E_{\mathfrak{F}}$ itself, while the weakest necessary condition is true, so the phrase is doubly misleading for what we do; our sense is counterfactual necessity under $\preceq_\tau$.

A minimal cause is then a selected element

$$C^\star \in \mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$$

that is $\lhd$-minimal: no valid cause is strictly preferred to it under the explicitly chosen preference order $\lhd$. Several incomparable minimal causes may exist, so we speak of *a* minimal cause rather than *the* minimal cause; $\lhd$ is assumed well-founded, so that whenever the valid-cause set $\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$ is non-empty at least one $\lhd$-minimal cause exists.

The intended output is not merely the statement that the controller wins. The intended output is a temporal explanation of what had to remain true in order for the chosen effect to occur.

2 Related Work and Contributions

This work sits at the intersection of three lines of research, and our claims should be read as deltas relative to them rather than as a fresh start.

Counterfactual and actual causation. The counterfactual-necessity core is the Lewis–Stalnaker tradition: a cause is what, on the closest worlds where it is absent, removes the effect, with centering and a limit assumption on the similarity order. Halpern–Pearl actual causation refines this with a minimal-sufficient-set requirement and a contingency quantifier; we deliberately adopt only the necessity half (Section 10) and are explicit about what this excludes (preemption discrimination, sufficiency).

Temporal/ω-regular causality. The closest technical relatives are automata-theoretic accounts of causes for ω-regular effects in reactive systems, notably the line of work synthesizing ω-regular causes for ω-regular effects (downward-closed causes via game/automaton constructions), which also underlies the ground truth of the TempoBench benchmark we target. Relative to that work, we do *not* claim a new causality-synthesis algorithm; our contribution is the *framing* layer below.

Reactive synthesis and parity games. We use only standard facts: LTL $\to$ deterministic parity automaton translation, parity games being determined and solvable in quasi-polynomial time (and in UP $\cap$ co-UP), and HOA as the interchange format for ω-automata. These are background, not contributions.

Contributions. Against this backdrop, the specific claims of this document are: (1) an explicit *causal frame* $\mathfrak{F}$ that factors a temporal causal query into background, effect scope, intervention model, candidate vocabulary, similarity order, and preference order, together with canonical default frames (Section 8.1) that pin all components but the declared vocabulary, grammar, and tie-breaks; (2) a clean separation of *trace-existential* necessity (one-player shortest-edit plus nonemptiness) from *strategy-guarantee* necessity (a two-player reverse parity game), with the latter

formalized and given a soundness statement (Section 12); (3) a precise, decidable anti-circularity condition (Assumption 2) with a mandatory effect cone (Assumption 3) for general effects, barring logical entailment and point-position restatement; and (4) a finite-trace specialization that aligns the framework with TempoBench-style TCE (Appendix D). The novelty is organizational and definitional — a reusable problem statement and its regimes — not a new hardness result or solver. We flag this explicitly so the contribution is judged as such.

3 Background: What a Parity Game Does

A parity game is a mathematical model of an infinite interaction between two players. In synthesis, the two players are usually:

- the environment, which chooses inputs, and
- the system, which chooses outputs.

Formally, a parity game is a tuple

$$G = (V, V_0, V_1, T, \Omega),$$

where V is the set of vertices, partitioned into the system vertices V_0 and the environment vertices V_1 with $V = V_0 \cup V_1$ and $V_0 \cap V_1 = \emptyset$; $T \subseteq V \times V$ is the set of edges; and $\Omega : V \to \mathbb{N}$ assigns a priority to each vertex. We require the edge relation to be *total*, i.e.

$$\forall v \in V \; \exists v' \in V. \; (v, v') \in T,$$

so that every vertex has a successor and every play extends to an infinite path (dead ends are excluded). The game is played on this directed graph. Each vertex belongs to one of the two players. When the token is on an environment vertex, the environment chooses the next edge. When the token is on a system vertex, the system chooses the next edge. Since reactive systems are meant to run forever, a play is an infinite path through this graph:

$$v_0 v_1 v_2 \cdots .$$

The word "parity" refers to the winning condition. Each vertex has a priority, which is a natural number. During an infinite play, some priorities may appear only finitely many times and some may appear infinitely often. In this document we use the following convention:

the system wins iff the least priority seen infinitely often is even.

This convention is arbitrary but standard; some texts use the greatest priority seen infinitely often instead. What matters is that the convention is fixed.

Parity games are useful for LTL synthesis because LTL formulas can be translated into deterministic parity automata. The automaton tracks whether the infinite input/output trace satisfies the formula. The parity game combines this automaton with choices made by the environment and the system. Solving the game answers the following yes/no question: does there exist a system strategy σ such that, for every environment behavior ρ, the play that is induced by σ and ρ satisfies φ? Formally:

$$\exists \sigma \; \forall \rho \; . \; \mathrm{Play}(\sigma, \rho) \models \varphi.$$

If the answer is yes, the strategy can be implemented as a finite-state controller, often represented as a Mealy machine.

4 The Forward View

Let

$$M = (S, s_0, I, O, \delta, \lambda)$$

be a Mealy machine. The set S contains the internal states of the machine, and $s_0 \in S$ is the initial state. The set I contains input atomic propositions controlled by the environment, and O contains output atomic propositions controlled by the system.

At time t, the environment chooses a set

$$i_t \in 2^I$$

of inputs that are true at that time. The Mealy machine responds with a set

$$o_t = \lambda(s_t, i_t) \in 2^O$$

of outputs that are true at that time, and then updates its state by

$$s_{t+1} = \delta(s_t, i_t).$$

The visible behavior at time t is the combined input/output valuation

$$a_t = i_t \cup o_t \in 2^{I \cup O}.$$

The resulting trace is the infinite word

$$\tau = a_0 a_1 a_2 \cdots .$$

This model explains why the forward direction loses explanatory detail. The machine tells us which output follows from each state and input, but it does not tell us which temporal property should be regarded as the reason for an observed effect. For example, the same machine might satisfy both a request response property and a weaker eventual-grant property. Looking only at the machine does not determine which of those properties is the intended cause.

5 The Reverse Question

The reverse construction proposed here is not

$$M \longrightarrow \varphi.$$

Instead, it is

$$M + \varphi + \mathfrak{F} + E + \tau \longrightarrow \text{minimal causal temporal property } C.$$

In the rest of this document, φ keeps its ordinary synthesis meaning: it is an LTL specification over $I \cup O$. The reverse-causal construction introduces a separate object, the causal frame $\mathfrak{F}$. The frame records how the explanation problem is scoped: which effect is selected, which assumptions are background, which propositions are relevant, which candidate causes are allowed, and which counterfactual interventions are admissible. When the frame is derived from an original specification, we write

$$\mathfrak{F} = \mathrm{Frame}(\varphi; \eta),$$

where η denotes the additional choices needed for causal explanation.

The formula φ may describe the original specification, but $\mathfrak{F}$ supplies the reverse-causal structure, and E is the effect under explanation, while $\mathfrak{F}$ may further restrict it to a scoped target $E_{\mathfrak{F}}$ (Section 8). Typical effects include:

- an output eventually occurs after a request,
- a particular transition or state recurrence happens,
- the observed trace satisfies a chosen subproperty,
- the product of the machine and an automaton remains in an accepting region.

Choosing E is essential. If M realizes all of φ against every environment, then asking what caused φ may be too broad. A sharper question is: what caused this effect on this execution, or what rule embedded in this strategy is responsible for this class of effects?

The reverse parity game asks a counterfactual explanation question:

$$\begin{gathered}\text{For a candidate cause } C,\\ \text{do all closest counterfactual plays violating } C\\ \text{make the effect } E \text{ fail?}\end{gathered}$$

This is different from ordinary synthesis. Synthesis searches for a strategy that forces a specification to hold. Reverse explanation starts with a fixed strategy and asks which temporal condition is necessary for the chosen effect.

6 Step-by-Step Construction

Throughout this section, E denotes the frame-selected effect: when the frame scopes the effect we mean $E_{\mathfrak{F}}$, and when no scoping is used $E_{\mathfrak{F}} = E$. We write E for brevity and switch to the explicit $E_{\mathfrak{F}}$ in the formal development of Section 8.

6.1 Step 1: Freeze the System Strategy

First compile the effect E, or the relevant part of the original specification, into a deterministic parity automaton

$$A_E = (Q, q_0, 2^{I \cup O}, \delta_E, \Omega).$$

The automaton reads one input/output valuation at a time. Its state records the progress of the effect, and its priority function Ω determines whether the infinite trace satisfies E. We use the same parity convention for A_E as for games (Section 3): a run is accepting iff the least priority occurring infinitely often is even, and priorities are assigned to states (state-based acceptance); a transition-based variant assigns Ω to edges instead and changes nothing essential below.

Now form the synchronous product

$$P = M \times A_E.$$

A product state has the form

$$(s, q),$$

where $s \in S$ is a Mealy-machine state and $q \in Q$ is an automaton state. From product state (s_t, q_t), one transition proceeds as follows:

1. The environment chooses $i_t \in 2^I$.
2. The Mealy machine produces $o_t = \lambda(s_t, i_t)$.

3. The Mealy machine updates to $s_{t+1} = \delta(s_t, i_t)$.

4. The automaton updates to
$$q_{t+1} = \delta_E(q_t, i_t \cup o_t).$$

The important change is that the system is no longer a free player. Its choices have already been fixed by M. Consequently the *trace-level* necessity test — searching the admissible set for the closest $\neg C$-traces and asking whether they preserve E — is a one-player shortest-edit search plus non-emptiness/reachability check in *all three* intervention regimes, including strategy-structure: freezing M removes the system's free move from this search. A genuine *two-player* parity game arises only for the distinct *strategy-guarantee* question (notion N2 of Section 12.1): may the system *re-open* its commitments and still force E against every environment? That question is posed over the original synthesis game G_φ with σ partially re-opened, and we reserve the term *reverse parity game* for it (Section 12).

It is therefore cleanest to keep two distinct arenas in mind. The frozen product $P = M \times A_E$ introduced here is a *one-player* arena: it underlies the trace-level necessity test (N1) in all regimes, where the system has no free moves and the question is a shortest-edit search plus non-emptiness/reachability. The genuinely two-player arena is the original synthesis game G_φ with the winning strategy σ partially re-opened (Step 6.6, "causal dominion extraction"); it underlies the strategy-guarantee question (N2). These are not the same object, and we are careful below not to present a two-player game over the frozen product P.

6.2 Step 2: Choose the Actual Trace

Causality is evaluated relative to an observation. Let
$$\tau = (i_0 \cup o_0)(i_1 \cup o_1)(i_2 \cup o_2)\cdots$$
be the actual trace generated by M. In implementations, τ is often stored as a lasso
$$uv^\omega,$$
where u is a finite prefix and v is a finite loop that repeats forever.

A candidate cause must be true on the actual trace, and the effect must also be true on the actual trace:
$$\tau \models C \qquad \text{and} \qquad \tau \models E.$$
This condition prevents vacuous explanations. A property that was not present in the observed execution cannot explain that execution.

6.3 Step 3: Define the Allowed Counterfactuals

A counterfactual trace is a trace that differs from the actual trace in a controlled way. The construction depends on which changes are allowed. This step does not define φ. It defines or instantiates $\mathcal{I}_\mathfrak{F}$, the intervention component of the causal frame. The same original LTL formula φ can support different reverse-causal questions depending on whether $\mathfrak{F}$ allows input-only, output-intervention, or strategy-structure counterfactuals.

Input-only counterfactuals. The environment may change inputs, but the Mealy machine must respond according to its transition and output functions. Counterfactual traces therefore remain inside the language $\mathcal{L}(M)$ generated by M. This is appropriate when the question is: which environment behavior caused the machine to produce the effect?

Output-intervention counterfactuals. The explanation game may allow direct changes to outputs. Such interventions can break the ordinary behavior of M, because the changed output may not be the output prescribed by λ. This version asks which system decisions were necessary for the effect, but it no longer restricts attention to genuine executions of the original machine.

Strategy-structure counterfactuals. The game may allow changes to strategy commitments, transitions, priorities, or winning-region choices in the parity-game representation. This is the most parity-game-native version. It asks which commitments of the winning strategy prevent the environment from reaching a losing region. The resulting cause may be a temporal rule such as

$$G(req \to X\, pending)$$

or

$$G(pending \wedge ack \to X\, \neg pending).$$

These are not just single events. They are rules about how the strategy must continue to behave over time.

Admissible counterfactuals per regime. Each intervention model $\mathcal{I}_{\mathfrak{F}}$ fixes the admissible counterfactual set $\mathrm{Adm}_M^{\mathfrak{F}}(\tau)$, the traces the reverse procedure is allowed to consider. The three regimes instantiate it differently:

- **Input-only:** counterfactuals must remain genuine machine behaviors, so

$$\mathrm{Adm}_M^{\mathfrak{F}}(\tau) = \mathcal{L}(M) \cap \mathcal{L}(B_{\mathfrak{F}}).$$

- **Output-intervention:** outputs may be edited away from λ, so admissible traces leave $\mathcal{L}(M)$. A natural choice is

$$\mathrm{Adm}_M^{\mathfrak{F}}(\tau) = \mathrm{Edit}_O^{\mathfrak{F}}(\mathcal{L}(M)) \cap \mathcal{L}(B_{\mathfrak{F}}),$$

  where $\mathrm{Edit}_O^{\mathfrak{F}}$ is the set of traces obtained from $\mathcal{L}(M)$ by the output edits the frame permits. Schematically, fix a frame-permitted set $P_{\mathfrak{F}} \subseteq \mathbb{N} \times O$ of (timestep, output) positions that may be toggled; then

$$\mathrm{Edit}_O^{\mathfrak{F}}(\mathcal{L}(M)) = \{\, \tau' \mid \exists\, \tau'' \in \mathcal{L}(M).\ \tau' \text{ and } \tau'' \text{ agree off } P_{\mathfrak{F}} \,\},$$

  i.e. τ' is obtained from some genuine machine behavior by flipping only outputs at positions in $P_{\mathfrak{F}}$. Inputs are untouched, so a frame that permits no positions recovers $\mathcal{L}(M)$.

- **Strategy-structure:** the system's commitments are re-opened, so admissibility is defined over plays consistent with some strategy in the frame-permitted neighborhood $\Sigma_{\mathfrak{F}}$ of the frozen strategy:

$$\mathrm{Adm}_M^{\mathfrak{F}}(\tau) = \{\, \mathrm{Play}(\sigma', \rho) \mid \sigma' \in \Sigma_{\mathfrak{F}},\ \rho \text{ an environment behavior} \,\} \cap \mathcal{L}(B_{\mathfrak{F}}).$$

  Schematically, $\Sigma_{\mathfrak{F}}$ is a set of strategies that differ from the frozen strategy σ (the one implemented by M) on a frame-bounded set of game vertices, e.g. all σ' agreeing with σ outside a designated editable vertex set $V_{\mathfrak{F}} \subseteq V$. By default $\Sigma_{\mathfrak{F}}$ ranges over *finite-memory* strategies on G_φ (sufficient for ω-regular objectives on finite arenas), so a witness deviation can always be taken to be a finite-memory strategy realizing an accepting lasso (Section 12). We require $\sigma \in \Sigma_{\mathfrak{F}}$, which guarantees that the actual trace is itself admissible ($\tau \in \mathrm{Adm}_M^{\mathfrak{F}}(\tau)$); see Assumption 1 below.

The input-only case is the default used in the rest of the document unless stated otherwise. Crucially, a candidate that all admissible traces already satisfy cannot be a non-vacuous cause in that regime; see the remark on strategy-level causes in Sections 7 and 8.

Two of these three regimes do not in fact depend on the actual trace τ: in the input-only and output-intervention cases $\mathrm{Adm}^{\mathfrak{F}}_M$ is a fixed language. We nonetheless write $\mathrm{Adm}^{\mathfrak{F}}_M(\tau)$ uniformly because frame-relative interventions *can* restrict to perturbations of τ itself; the finite-horizon TempoBench specialization of Appendix D, which perturbs only $\tau_{\le k}$, is exactly such a case.

6.4 Step 4: Define Similarity to the Actual Trace

Counterfactual causality requires a notion of closeness. Let

$$\preceq_\tau$$

be a preorder over traces, where

$$\tau' \preceq_\tau \tau''$$

means that τ' is at least as close to the actual trace τ as τ'' is.

The primary closeness order used throughout this document is the *minimal-intervention lexicographic* order. For a fixed actual trace τ and a trace τ', let the *change set* be

$$\mathrm{Diff}_\tau(\tau') = \{t \in \mathbb{N} \mid \tau'(t) \neq \tau(t)\}, \qquad d(\tau, \tau') = |\mathrm{Diff}_\tau(\tau')| \in \mathbb{N} \cup \{\infty\},$$

with the input-only variant d_I counting only positions whose *input* valuation differs. Fix once and for all an arbitrary total order $\lhd_\Sigma$ on the finite alphabet $2^{I \cup O}$, and for a trace with finite change set write $\mathrm{prof}_\tau(\tau')$ for the *difference profile*: the finite list $\langle (t_1, \tau'(t_1)), \ldots, (t_d, \tau'(t_d)) \rangle$ of changed positions in increasing position order, each tagged with its new value. Define

$$\tau' \preceq_\tau \tau'' \quad \Longleftrightarrow \quad \big(d(\tau, \tau'),\ \mathrm{prof}_\tau(\tau')\big) \ \le_{\mathrm{lex}} \ \big(d(\tau, \tau''),\ \mathrm{prof}_\tau(\tau'')\big),$$

comparing first the change counts (with ∞ above every natural number), then, for equal finite counts, the difference profiles lexicographically (position first, then value under $\lhd_\Sigma$). Traces with an *infinite* change set all share the top count ∞; we do *not* attempt to order them among themselves and instead treat them as a single $\preceq_\tau$-maximal class that is never selected as a closest removal when any finite-change removal exists. Accordingly we define $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C)$ over the *finite-change* removals (those with $d < \infty$) whenever that set is non-empty.

On the finite-change traces this $\preceq_\tau$ is a genuine *total order* (any two are comparable, and equal lex-keys force equal traces because the profile pins every changed position to its value and τ pins the rest), so in particular it is antisymmetric — the hypothesis Proposition 1 uses. Its unique minimum is τ itself, since $d(\tau, \tau) = 0$; this is the centering condition (Assumption 1(ii)). It is *well-founded on the finite-change traces*: the count key ranges in the well-ordered $\mathbb{N}$, and within a fixed finite count d the profiles are length-d tuples over $\mathbb{N} \times 2^{I \cup O}$ ordered lexicographically, which is well-ordered, so there are no infinite strictly descending chains (Assumption 1(iii)). We stress that the *lexicographic comparison of infinite-change profiles would not be well-founded* — it admits infinite descending chains — which is exactly why Close is restricted to finite-change removals; for effects whose only removals require infinitely many edits, Close is empty and the candidate fails non-vacuity, the correct verdict under non-vacuity. This is a **scope boundary**, stated explicitly: trace-level necessity (N1) is confined to *co-safety / finite-change* causes. A genuine ω-liveness cause — one whose removal requires infinitely many edits, e.g. removing $GF\,req$ from a lasso forces $FG\,\neg req$ — has Close $= \emptyset$ and is reported as non-vacuously failing at the trace level; such causes

belong to the strategy level (N2), where "dropping a commitment" is a single strategic deviation rather than infinitely many trace edits (Section 12). We make the count the *primary* key on principled grounds: it is the standard *minimal-intervention* / Hamming-distance notion for trace repairs (fewest changed positions first), not a choice tuned to any one example. As an illustration of its effect, in the toy example of Section 9 "issue the request one step later" changes strictly more positions than the minimal removal, hence has a larger count and is $\preceq_\tau$-above it, and so does not enter $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg req)$. The optional *profile* tie-break that orders same-count removals can make validity hinge on an implementation detail (which minimum-count removal is selected), so reports that use it must declare it. The recommended default is therefore the *count-only coarsening* (drop the profile tie-break), under which Close is the full set of minimum-count removals rather than a single trace and validity quantifies over *all* of them, removing the tie-break dependence; this is also the order the TempoBench specialization of Appendix D uses, since temporal causal credit assignment asks for necessity over *all* minimal-change removals.

We deliberately do *not* take the first-difference order as primary. Comparing traces only by the earliest time they differ from τ is the order induced by the *Baire metric* (an ultrametric) on the Cantor space $(2^{I\cup O})^\omega$ of infinite words: two traces are closer the later they first differ. That order is only a *preorder* (all traces that first differ at the same position are tied), it is not antisymmetric, and under it the later-request counterfactual above would be *tied* with the minimal removal and would spuriously defeat the intended cause. The pure change-set *inclusion* order has the dual defect: the minimal removal (edit $\{(0,\emptyset),(1,\emptyset)\}$) and the later-request trace (edit $\{(0,\emptyset),(1,\{req\}),(2,\{grant\})\}$) have incomparable edit sets, so both would be minimal and the cause would again fail. The count-based order above avoids both pathologies; Proposition 1 holds for any well-founded $\preceq_\tau$ (count-only or count-first), and the toy example is stated under it.

Remark 1 (Closest removals are well-behaved for lasso counterfactuals). When τ is a lasso uv^ω, M is finite-state, and the regime is input-only or output-intervention, every admissible $\neg C$-trace is itself a lasso of bounded period (a path in the finite product $M \times A_{\neg C} \times A_{\preceq}$), and the change count d is finite exactly on the eventually-periodic perturbations that agree with τ on its tail. Hence the candidate set $\{\tau' \in \mathrm{Adm}^{\mathfrak{F}}_M(\tau) \mid \tau' \models \neg C\}$ has a minimum-count sublevel that is finite up to lasso representation. Under the count-only default $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C)$ is the full set of minimum-count removals, computable by a BFS/shortest-edit search over that product; under the optional profile refinement (a total order) it is the unique $\preceq_\tau$-least element, a single shortest-edit witness. Either way "closest removal" is finitely represented and computable rather than a large, divergent set.

Remark 2 (Centering and the strategy-structure regime). Strategy-guarantee necessity (N2, Section 12.1) is a *type-level* property of the controller σ — "can σ drop a commitment and still guarantee E?" — and is centered on the actual *strategy* σ, not on the actual trace τ. It therefore does *not* require trace-centering (Assumption 1(ii)), and Proposition 3 does not use it; the trace-level Lewis–Stalnaker axiom is simply not the right notion at the strategy level. Trace-centering matters only for the *token-level* certificate $C^\flat$ (Proposition 1): if one wishes to run that construction *within* a strategy-structure frame (a separate, optional use), the edited-vertex subset order ties all plays of σ with τ, so one must refine it lexicographically (edited-vertex subset, then the trace-level count-first order) to restore τ as unique minimum. Absent that refinement, Proposition 1 is read only for the input-only/output regimes, while the strategy-structure regime carries the validity content of Proposition 3 (which needs no centering).

Given a candidate cause C, define the closest counterfactuals that remove C as

$$\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) = \min_{\preceq_\tau}\{\tau' \in \mathrm{Adm}^{\mathfrak{F}}_M(\tau) \mid \tau' \models \neg C\}.$$

This is the set of admissible counterfactual traces allowed by $\mathfrak{F}$ that violate C and are minimal under the chosen closeness order, with $\mathrm{Adm}^{\mathfrak{F}}_M(\tau)$ instantiated per regime as in Step 6.3.

We collect the standing well-formedness hypotheses that several later arguments silently rely on.

Assumption 1 (Well-formedness of the counterfactual model)**.** *Fix M, $\mathfrak{F}$, and the actual trace τ. We assume:*

(i) ***Actual-world admissibility.*** *$\tau \in \mathrm{Adm}^{\mathfrak{F}}_M(\tau)$; the actual trace is itself an admissible world. (In the input-only regime this means $\tau \in \mathcal{L}(M) \cap \mathcal{L}(B_{\mathfrak{F}})$, so in particular $\tau \models B_{\mathfrak{F}}$; in the strategy-structure regime it follows from $\sigma \in \Sigma_{\mathfrak{F}}$.)*

(ii) ***Centering.*** *$\preceq_\tau$ has τ as its unique minimum: $\tau \prec_\tau \tau'$ for every admissible $\tau' \neq \tau$. The actual world is strictly closest to itself.*

(iii) ***Well-foundedness (Limit Assumption).*** *$\preceq_\tau$ is well-founded on $\mathrm{Adm}^{\mathfrak{F}}_M(\tau)$: there is no infinite strictly descending chain $\cdots \prec_\tau \tau_2 \prec_\tau \tau_1$. Consequently every non-empty admissible subset — in particular $\{\tau' \in \mathrm{Adm}^{\mathfrak{F}}_M(\tau) \mid \tau' \models \neg C\}$ when it is non-empty — has $\preceq_\tau$-minimal elements, so that $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C)$ is non-empty in that case; and moreover every admissible element lies above some $\preceq_\tau$-minimal element of any admissible set that contains it.*

Parts (i)–(ii) are the standard centering conditions of Lewis–Stalnaker counterfactual semantics; they ensure that conditioning on $\neg C$ is evaluated against worlds genuinely reachable under the frame and that the actual trace is the reference point of the similarity order. The two consequences drawn in part (iii) — existence of minimal removals and domination of each removal by a minimal one — are exactly what the non-vacuity clause and Proposition 1(c) rely on. Well-foundedness holds automatically when the candidate space and admissible set are finitely representable (for example in the finite-horizon TempoBench-compatible setting of Appendix D), and it holds for the canonical count-first lexicographic order of Step 6.4 *when* Close *is restricted to finite-change removals* (the count key is then well-ordered and, at each finite count, the difference profiles are; see Remark 1 for the lasso case). We emphasize that on infinite-change traces the lexicographic profile comparison is *not* well-founded, so part (iii) is genuinely a restriction to finite-change removals (or, for orders other than the canonical one, an explicit hypothesis on $\preceq_\tau$), not a free lunch.

6.5 Step 5: Check a Candidate Cause (One-Player Test)

Given a candidate C, build automata for C, $\neg C$, and E. The counterfactual adversary tries to find a trace satisfying

$$\tau' \in \mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) \cap \mathcal{L}(E).$$

The intersections with $\mathcal{L}(M)$ and $\mathcal{L}(\neg C)$ are not written here because $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) \subseteq \mathrm{Adm}^{\mathfrak{F}}_M(\tau)$ and already consists of traces satisfying $\neg C$; in the input-only regime that already entails $\tau' \in \mathcal{L}(M)$. Such a trace is a counterexample to the explanation: it says that C can be removed while E still holds on a closest allowed counterfactual.

This trace-existential test (notion N1 of Section 12.1) is a *one-player* shortest-edit search plus non-emptiness check in *every* regime: once M is frozen the system has no free move, and the adversary merely searches the admissible language for the closest $\neg C$-traces and asks whether any preserves E. The genuinely two-player reverse parity game arises only for the distinct *strategy-guarantee* notion (N2), treated in Section 12, not for this trace test. The explainer wins when the adversary cannot find such a trace. Equivalently, C passes the counterfactual test when

$$\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) \neq \emptyset \quad \text{and} \quad \mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) \cap \mathcal{L}(E) = \emptyset.$$

The first conjunct is the non-vacuity requirement: there must actually be a closest way to remove C. Thus a candidate cause is valid exactly when removing it is realizable and destroys the effect on all closest allowed counterfactual executions.

6.6 Step 6: Search for a Minimal Cause

Checking one candidate is not enough. The real goal is to find the best candidate according to an explicit preference order. Three useful approaches are available.

Automata-theoretic cause synthesis. When the effect is ω-regular and the similarity relation is regular enough to be represented automata-theoretically, the cause can itself be represented by an automaton. Write

$$A = \mathrm{Adm}^{\mathfrak{F}}_M(\tau), \qquad F = \{\tau' \in A \mid \tau' \not\models E_{\mathfrak{F}}\}$$

for the admissible set and the admissible *effect-failures*, and recall $\uparrow_{\preceq_\tau}(F) = \{\tau'' \mid \exists\, \tau' \in F.\ \tau' \preceq_\tau \tau''\}$. The direction of the preorder must be read carefully: because $\tau' \preceq_\tau \tau''$ means τ' is *closer* to the actual trace τ, the upward closure $\uparrow_{\preceq_\tau}(F)$ is the set of traces that are *at least as far* from τ as some effect-failure. "Above" means further away, not closer or better. The synthesized cause $C^\flat$ is *any* ω-regular property whose intersection with the admissible set is the failure-free neighborhood,

$$\mathcal{L}(C^\flat) \cap A = \overline{\uparrow_{\preceq_\tau}(F)} \cap A = A \setminus \uparrow_{\preceq_\tau}(F),$$

i.e. $\tau' \models C^\flat$ (within A) exactly when no admissible effect-failure is at least as close to τ as τ'. Note that this pins $C^\flat$ only on A; off A it is unconstrained, so "$C^\flat$" names an equivalence class of properties agreeing on A, not a unique formula. We speak of *a* canonical certificate, and the definite article below is shorthand for "any representative."

Proposition 1 (Nearest-failure-anchored cause)**.** *Suppose Assumption 1 holds, that $\preceq_\tau$ is a partial order on A (antisymmetric, as the count-first lexicographic profile refinement of Step 6.4 is, being total), that the actual effect holds ($\tau \models E_{\mathfrak{F}}$), and that there is at least one admissible effect-failure, $F \neq \emptyset$. Then:*

(a) $C^\flat$ satisfies the four valid-cause clauses of $\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$ (actuality, the two effect clauses, and non-vacuity); hence $C^\flat$ is a valid cause provided it is admitted to $\mathcal{K}_{\mathfrak{F}}$. *As a distance-defined certificate, $C^\flat$ need not itself lie in $\mathcal{K}_{\mathfrak{F}}$ (it may be unreadable or fail effect-disjointness), so we do* not *assert $C^\flat \in \mathsf{Causes}$ unconditionally; $C^\flat$ is a witness object, not necessarily the returned answer $C^\star$;*

(b) its closest removals are exactly the globally nearest admissible effect-failures,

$$\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C^\flat) = \min_{\preceq_\tau}(F);$$

(c) among valid causes whose closest removals are the globally nearest effect-failures, $C^\flat$ is the smallest-language *one: $\mathcal{L}(C^\flat) \cap A \subseteq \mathcal{L}(C) \cap A$ for every such C. Equivalently — and this is not a contradiction — its admissible models $\mathcal{L}(C^\flat) \cap A$ form the* largest *failure-free neighborhood of τ (the set of admissible traces strictly closer to τ than every admissible effect-failure), because the complement $\uparrow_{\preceq_\tau}(F)$ it excludes is the* smallest *failure-containing up-set; the cause is language-minimal precisely because the failures it must admit are pushed as far out as possible.*

Proof. Within A we have $\neg C^\flat \equiv \uparrow_{\preceq_\tau}(F)$, so the admissible $\neg C^\flat$-traces are exactly $\uparrow_{\preceq_\tau}(F)$.

(b) and minimality of failures. We first show $\min_{\preceq_\tau}(\uparrow(F)) = \min_{\preceq_\tau}(F)$. Since $F \subseteq \uparrow(F)$, take $m \in \min(F)$; if some $x \in \uparrow(F)$ had $x \prec_\tau m$, then $x \in \uparrow(F)$ gives $f \preceq_\tau x$ for some $f \in F$, whence $f \preceq_\tau m$; were $m \preceq_\tau f$ we would get $m \preceq_\tau x$, contradicting $x \prec_\tau m$, so $f \prec_\tau m$, contradicting $m \in \min(F)$. Thus $m \in \min(\uparrow(F))$, giving $\min(F) \subseteq \min(\uparrow(F))$. Conversely, if $m \in \min(\uparrow(F))$ then $f \preceq_\tau m$ for some $f \in F \subseteq \uparrow(F)$; minimality of m forces $m \preceq_\tau f$, and antisymmetry gives $m = f \in F$, so $m \in \min(F)$. Hence the two minimal sets coincide, and $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C^\flat) = \min(\uparrow(F)) = \min(F)$, which proves (b).

(a). By the Limit Assumption and $F \neq \emptyset$, $\min(F) \neq \emptyset$, so $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C^\flat) \neq \emptyset$ (non-vacuity). By (b) every closest removal lies in F, so it fails $E_{\mathfrak{F}}$; hence $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C^\flat) \cap \mathcal{L}(E_{\mathfrak{F}}) = \emptyset$. Actuality holds: $\tau \models E_{\mathfrak{F}}$ by hypothesis, and $\tau \models C^\flat$ because by centering (Assumption 1(ii)) no $f \in F$ (each $f \neq \tau$) satisfies $f \preceq_\tau \tau$, so $\tau \notin \uparrow(F)$. All four valid-cause clauses hold; whether $C^\flat \in \mathsf{Causes}$ then depends only on whether $C^\flat \in \mathcal{K}_{\mathfrak{F}}$, as noted in (a).

(c). Let C be any valid cause with $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) = \min(F)$, and suppose $\tau' \in \mathcal{L}(C^\flat) \cap A$, i.e. $\tau' \notin \uparrow(F)$: no effect-failure is at least as close as τ'. If $\tau' \notin \mathcal{L}(C)$, then τ' is an admissible $\neg C$-trace, so by well-foundedness (Assumption 1(iii)) it lies above some $\preceq_\tau$-minimal $\neg C$-trace, i.e. $\tau' \in \uparrow_{\preceq_\tau}(\mathrm{Close}(\neg C)) = \uparrow(\min(F)) \subseteq \uparrow(F)$ (some closest removal is at least as close as τ'), contradicting $\tau' \notin \uparrow(F)$. Hence $\mathcal{L}(C^\flat) \cap A \subseteq \mathcal{L}(C) \cap A$; since C was arbitrary in this class, $C^\flat$ has the smallest language among anchored valid causes, and $\mathcal{L}(C^\flat) \cap A = A \setminus \uparrow(F)$ is exactly the set of admissible traces strictly closer to τ than every admissible effect-failure. □

Remark 3 (Why "the weakest valid cause" is the wrong target). It is tempting to seek the *extensionally maximal* (largest-language) valid cause — we avoid calling it "weakest" to prevent clash with Dijkstra's weakest precondition — but that object is degenerate and non-explanatory. Removing a single nearest failure $f_0 \in \min_{\preceq_\tau}(F)$ already gives a valid cause with language $A \setminus \{f_0\}$: its only closest removal is f_0, which fails $E_{\mathfrak{F}}$. Such causes are essentially maximal yet say nothing, the choice of f_0 is arbitrary, and the valid-cause set therefore has no useful greatest element. The construction does *not* return this; the *answer* is the $\lhd$-minimal readable $C^\star$ (below), while $C^\flat$ is a *certificate* whose language is the maximal failure-free neighborhood of τ (Proposition 1(c)). $C^\flat$ is a τ-specific, distance-defined object — "stay strictly closer to τ than every way of losing the effect" — determined only on A, generally *not* a short or human-readable temporal formula, and not necessarily a member of $\mathcal{K}_{\mathfrak{F}}$; readable, preference-order-minimal explanations are obtained instead by the search routes below, which optimize $\lhd$ over $\mathcal{K}_{\mathfrak{F}}$. The certificate stays ω-regular when $\uparrow_{\preceq_\tau}(F)$ is ω-regular — which holds for the count-first order via the shortest-edit construction of Remark 1 on lassos, and otherwise must be posited as an explicit automaticity hypothesis on $\preceq_\tau$ (we do not claim the order *relation* is automatic in general; see Section 13). Antisymmetry of $\preceq_\tau$ is what the proof uses; *totality* is sufficient but not necessary, so the guarantee holds for the count-first lexicographic profile refinement of Step 6.4 (which is total, hence antisymmetric) as well as for any merely antisymmetric partial order; under the count-only default it holds via the nearest-layer invariance hypothesis below. For a non-antisymmetric preorder such as the first-difference (Baire) order the same argument goes through only under the extra hypothesis that $E_{\mathfrak{F}}$ is invariant under $\preceq_\tau$-equivalence on the nearest layer (all traces tied with a nearest failure also fail $E_{\mathfrak{F}}$); without that hypothesis the conclusion can fail, which is one reason we do not adopt the Baire order as primary.

Counterexample-guided search over LTL sketches. Another option is to restrict explanations to a readable grammar, such as

$$C ::= p \mid \neg C \mid C \wedge C \mid XC \mid FC \mid GC \mid C\,U\,C,$$

or to a smaller template family such as

$$G(a \to Fb), \qquad F(a \wedge Xb), \qquad G(a \to Xb), \qquad GFa, \qquad FGa.$$

The loop is:

1. propose a candidate C,
2. check it in the reverse parity game,
3. if it fails, extract a counterfactual trace τ',
4. refine the candidate,
5. minimize according to the chosen ordering.

This approach is practical when the output should be a human-readable LTL formula rather than only an automaton.

Causal dominion extraction. A more parity-game-native approach begins with the original parity game G_φ and the winning strategy σ. Restrict the game to plays consistent with σ, then ask which strategy commitments prevent entry into losing dominions. The extracted cause may be a set of vertices

$$D_C \subseteq V$$

or a set of guarded transitions

$$T_C \subseteq T.$$

The temporal explanation is then obtained by translating those commitments into conditions over traces. For example, a commitment may become the rule

$$G(req \to X\, pending),$$

because without that rule the environment can force the play into a region where requests remain unresolved.

7 Trace Causes and Strategy Causes

There are two related but distinct explanation targets.

Trace-level cause. A trace-level cause explains one observed execution. Given an actual trace τ, it finds a property C_τ that caused E on that trace. For example, if the actual trace contains a request followed immediately by a grant, then a trace-level explanation may be

$$C_\tau = req \wedge X\, grant.$$

This says what happened on that execution.

Strategy-level cause. A strategy-level cause explains why the controller as a whole realizes a property. Given M, it finds a temporal rule C_M that is satisfied by all traces of the machine and is responsible for the effect or specification. A typical strategy-level explanation is

$$C_M = G(req \to F\, grant).$$

This says what the controller must keep doing across all relevant executions.

The strategy-level version is more ambitious. It is not merely asking which event caused another event. It asks which temporal invariant, recurrence rule, or response rule embedded in the controller is responsible for success.

Strategy-level causes require a non-input-only frame. The following makes precise why a strategy-level cause cannot be tested under an input-only frame.

Proposition 2 (Strategy-level causes need a re-opening frame)**.** *Let C_M be a strategy-level cause, i.e. an invariant of M with $\mathcal{L}(M) \subseteq \mathcal{L}(C_M)$. If $\mathfrak{F}$ permits only input-only counterfactuals, then C_M is never a valid cause: $C_M \notin \mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$ for every τ and $E_{\mathfrak{F}}$.*

Proof. Under the input-only frame $\mathrm{Adm}^{\mathfrak{F}}_M(\tau) = \mathcal{L}(M) \cap \mathcal{L}(B_{\mathfrak{F}}) \subseteq \mathcal{L}(M) \subseteq \mathcal{L}(C_M)$. Hence $\{\tau' \in \mathrm{Adm}^{\mathfrak{F}}_M(\tau) \mid \tau' \models \neg C_M\} = \emptyset$, so $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C_M) = \emptyset$ and the non-vacuity clause of $\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$ (Section 8) fails. □

Thus C_M is correctly rejected as vacuous rather than passing the necessity test trivially, and evaluating a strategy-level cause *requires* a frame whose intervention model $\mathcal{I}_{\mathfrak{F}}$ re-opens the machine's commitments so that some admissible counterfactual can violate C_M. The principled regime for this is *strategy-structure* intervention: its admissible traces are the plays of genuine alternative strategies $\sigma' \in \Sigma_{\mathfrak{F}}$, so a violation of C_M corresponds to a real strategic deviation and the test is a counterfactual over strategies. An *output-intervention* frame also produces traces outside $\mathcal{L}(M)$ and can therefore falsify C_M, but its edits need not correspond to any strategy; it is best understood as a separate output/event-causality regime that asks which individual output decisions were necessary, not as a test of the strategic rule C_M. We accordingly treat strategy-level causes as living in strategy-structure frames, and read output-intervention results as output causality. Input-only counterfactuals can meaningfully test only trace-level causes.

8 A Formal Definition

Let M be a Mealy machine and let $\tau \in \mathcal{L}(M)$ be the actual trace. The formula $\varphi \in \mathrm{LTL}(I \cup O)$ may be the original LTL specification, with semantics $\mathcal{L}(\varphi) \subseteq (2^{I \cup O})^\omega$. The reverse-causal object is the frame

$$\mathfrak{F} = (B_{\mathfrak{F}}, E_{\mathfrak{F}}, S_{\mathfrak{F}}, O_{\mathfrak{F}}, \mathcal{K}_{\mathfrak{F}}, \mathcal{I}_{\mathfrak{F}}, \preceq, \lhd).$$

Here $E_{\mathfrak{F}}$ is the effect selected by the frame and $S_{\mathfrak{F}}$ is the set of selected effect indices (used when $E = E_1 \wedge \cdots \wedge E_m$ is decomposed; see Appendix B). These two components are not independent: $E_{\mathfrak{F}} = \bigwedge_{r \in S_{\mathfrak{F}}} E_r$ is determined by $S_{\mathfrak{F}}$ and the decomposition of E, so listing both is a deliberate convenience (one may drop $E_{\mathfrak{F}}$ and keep $S_{\mathfrak{F}}$, or vice versa). Further, $B_{\mathfrak{F}}$ is the background constraint supplied by the frame, $O_{\mathfrak{F}} \subseteq O$ is the set of relevant outputs, $\mathcal{K}_{\mathfrak{F}}$ is the candidate-cause space, and $\mathcal{I}_{\mathfrak{F}}$ is the allowed intervention model (which fixes $\mathrm{Adm}^{\mathfrak{F}}_M$ as in Step 6.3). Environment assumptions A_{env} and system guarantees G_{sys}, when present, are folded into $B_{\mathfrak{F}}$. If there are no background constraints, take $B_{\mathfrak{F}} = \mathsf{true}$. The frame also supplies the family of similarity preorders $\preceq = \{\preceq_\tau\}_\tau$

(one per possible actual trace; the operative trace τ is a separate input that selects $\preceq_\tau$) and the candidate-minimality order $\lhd$, where $C' \lhd C$ means that C' is preferred to C.

First define the closest admissible counterfactuals that remove a candidate cause:

$$\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) = \min_{\preceq_\tau}\{\tau' \in \mathrm{Adm}^{\mathfrak{F}}_M(\tau) \mid \tau' \models \neg C\}.$$

The set $\mathrm{Adm}^{\mathfrak{F}}_M(\tau)$ contains the admissible counterfactual traces allowed by $\mathfrak{F}$. In the ordinary input-only case,

$$\mathrm{Adm}^{\mathfrak{F}}_M(\tau) = \mathcal{L}(M) \cap \mathcal{L}(B_{\mathfrak{F}}),$$

which does not in fact depend on τ; as noted in Step 6.3, the τ argument is retained uniformly only because frame-relative interventions (notably the TempoBench specialization of Appendix D) may restrict to perturbations of τ.

The set of valid causes is:

$$\begin{aligned}\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}}) = \{\, C \in \mathcal{K}_{\mathfrak{F}} \mid\ & \tau \in \mathcal{L}(C),\ \tau \in \mathcal{L}(E_{\mathfrak{F}}),\\ & \mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) \neq \emptyset,\\ & \mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) \cap \mathcal{L}(E_{\mathfrak{F}}) = \emptyset \,\}.\end{aligned}$$

The third conjunct is the non-vacuity requirement: there must be at least one admissible counterfactual that removes C. Without it, any C that no admissible trace violates, in particular a tautology or any invariant of M under an input-only frame, would satisfy the emptiness condition vacuously. Thus C_{valid} is a valid cause exactly when

$$C_{\mathrm{valid}} \in \mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}}).$$

The candidate space $\mathcal{K}_{\mathfrak{F}}$ must itself be constrained, or the necessity test admits circular "causes" that re-assert the effect. The key distinction is between *causing E through M* — which is exactly what a genuine cause does, and must be allowed — and *restating E* — asserting (part of) the effect at its own evaluation positions, which is circular. The condition below bans the latter without touching the former.

Assumption 2 (Anti-circularity of the candidate space)**.** *Every candidate $C \in \mathcal{K}_{\mathfrak{F}}$ is* effect-disjoint*, in two checkable senses:*

(i) ***No restatement (semantic, point/bounded effects).*** *For a point effect $E_{\mathfrak{F}} = X^k o$, C does not constrain the value of o at position k: $\mathcal{L}(C)$ is invariant under flipping o at position k (i.e. $\tau' \in \mathcal{L}(C) \iff \tau'[k{:}o{\mapsto}\neg o] \in \mathcal{L}(C)$, flipping only o at time k). This is the clean, semantic form of "C does not assert the effect at its own position"; it admits a cooldown cause "o at $t = 5$" for effect "o at $t = 10$" ($t \neq k$) and any inputs, while barring $X^k o$, $X^k \neg o$, and trivial re-encodings such as $X^k(o \vee \bot)$. For a bounded conjunctive effect the masking is applied at each constrained position.* For a general ω-regular effect *(GF grant, $G(req \to F\, grant)$, parity acceptance) there is no single "effect position" to mask, so guard (ii) and a* mandatory *declared effect cone (Assumption 3) apply instead.*

(ii) ***No pure-logic entailment (all effects).*** *C does not force the effect as a matter of logic* alone*, independent of M: $\mathcal{L}(C) \not\subseteq \mathcal{L}(E_{\mathfrak{F}})$. This is non-inclusion of two ω-regular languages, decidable by testing emptiness of $\mathcal{L}(C) \cap \overline{\mathcal{L}(E_{\mathfrak{F}})}$. Crucially the test is* not *relativized to $\mathrm{Adm}^{\mathfrak{F}}_M(\tau)$: a candidate that forces $E_{\mathfrak{F}}$ only* through the machine dynamics *(as every real cause does) is* not *excluded; only a candidate that forces $E_{\mathfrak{F}}$ by pure logic, i.e. that is already*

a logical strengthening of the effect, is. This clause is robust to trivial semantic re-encodings (e.g. $X^k(o \vee \bot) \equiv X^k o$ is barred) that a purely syntactic guard could miss, and it does the work for general effects, where (i) is silent.

We call this "precise and decidable" in the scoped sense that (i) is a decidable semantic invariance check for point/bounded effects, (ii) is a decidable ω-regular inclusion test for all effects, and the effect cone (Assumption 3) is a declared finite AP*-subset; we do* not *claim a uniform syntactic "effect-position" notion for arbitrary temporal effects, which is why (i) is scoped to point/bounded effects. Guards (i) and (ii) bar* logical entailment *and* point-position restatement*; they do not by themselves exclude system-relative aliases or near-restatements of a general effect, which is why the effect cone below is mandatory for such effects.*

Assumption 3 (Effect cone (mandatory for general effects))**.** *For every non-point effect $E_{\mathfrak{F}}$, the frame declares a finite* AP*-subset $\mathsf{Cone}(E_{\mathfrak{F}})$ of propositions that may appear in effect-shaped subformulas (outputs and acceptance atoms tied to E). Every $C \in \mathcal{K}_{\mathfrak{F}}$ must satisfy $\mathrm{AP}(C) \cap \mathsf{Cone}(E_{\mathfrak{F}}) = \emptyset$ unless C is explicitly marked, in $\mathfrak{F}_{\mathrm{strat}}$, as an internal-state observable (e.g. at_s) or a declared system-guarantee invariant. This blocks vacuous near-restatements that name the effect atom — e.g. a bare recurrence $GF\,grant$, or a guarded re-assertion of grant — for effect $GF\,grant$ when $\mathsf{Cone} = \{grant\}$, while still allowing input causes and, under a declared regime, strategy-level state observables and system-guarantee invariants (e.g. the controller commitment $G(req \to F\,grant)$, admitted only by such an explicit declaration, never at the trace level).*

This resolves what would otherwise be a fatal circularity in the wrong place. For the general effect $F\,grant$, guard (i) is silent and (ii) is operative: the input cause req satisfies $\mathcal{L}(req) \not\subseteq \mathcal{L}(F\,grant)$ (a trace with req at time 0 and no $grant$ ever is a model of req but not of $F\,grant$), so it is admitted. That req forces $F\,grant$ *on* $\mathcal{L}(M)$ is precisely why it is a cause, and the de-relativized (ii) does not penalize it. Likewise the TempoBench cause $\Gamma^\star$ (Appendix D), a set of input literals that pins o at k *through the functional machine*, satisfies $\mathcal{L}(C_{\Gamma^\star}) \not\subseteq \mathcal{L}(X^k o)$ and is admitted (for the point effect $X^k o$, guard (i) masks only $o@k$, which $\Gamma^\star$, being over inputs, does not touch); and the strategy-level invariant $G(req \to F\,grant)$ names the cone atom $grant$, so when $\mathsf{Cone}(F\,grant) = \{grant\}$ it is admitted *only* as a declared system-guarantee invariant under $\mathfrak{F}_{\mathrm{strat}}$ (never at the trace level); given that declaration it is not a logical restatement either, since a request-free, grant-free trace satisfies it vacuously while failing the effect. What is *excluded* is genuine restatement: $X\,grant$ and $req \wedge X\,grant$ for effect $F\,grant$ entail $F\,grant$ logically, failing (ii); a bare recurrence $GF\,grant$ or any candidate naming a cone atom without a declaration is barred by the cone; and $X^k o$, $X^k(o \vee \bot)$ for a point effect $X^k o$ are barred by (i) (and (ii)). Thus Assumption 2 together with the mandatory effect cone bars logical entailment, point-position restatement, and undeclared use of effect-cone atoms, independently of the preference order, while admitting all of the document's worked causes; it does not claim to exclude every system-relative alias, which is why a near-restatement naming a cone atom requires an explicit declaration.

Definition 1 (Minimal temporal cause)**.** *A candidate $C^\star \in \mathcal{K}_{\mathfrak{F}}$ is a minimal temporal cause of $E_{\mathfrak{F}}$ on τ relative to M, $\mathfrak{F}$, and $\lhd$ iff:*

1. ***Validity.***
$$C^\star \in \mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}}).$$
2. ***Minimality.*** *There is no $C' \in \mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$ such that*
$$C' \lhd C^\star.$$

The condition is $\lhd$-minimality, not a global minimum: when $\lhd$ is a partial order, several incomparable minimal causes may exist, and "minimal cause" names any one of them. We assume $\lhd$ is well-founded (which holds automatically when $\mathcal{K}_{\mathfrak{F}}$ is finite), so that whenever $\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$ is non-empty a minimal element exists.

The preference order $\lhd$ must be chosen deliberately, and its direction must be stated. Recall that validity already encodes necessity (Section 8); $\lhd$ only selects *among* necessary causes. An earlier draft of this work took semantic permissiveness (larger $\mathcal{L}(C)$, "logically weaker") as the *primary* key, but that choice is defective: if A and B are each valid necessary conditions and $A \vee B$ is too, then $A \vee B$ is strictly weaker than either, so a permissiveness-first order drifts toward ever-larger disjunctions of necessary conditions and would even prefer $F\,req$ to req. We therefore make *syntactic readability the primary key and permissiveness only a tie-break*. The canonical order is

$$\big(\text{smaller formula size},\ \text{smaller temporal depth},\ \text{more permissive (larger } \mathcal{L}(C))\big),$$

read lexicographically. For "formula size" and "temporal depth" to be well-defined and reproducible, we fix once a candidate grammar (the readable LTL grammar of Step 6.6) and a normal form (negation pushed to literals, a fixed associativity/commutativity ordering on $\wedge, \vee$, no redundant X-nesting), and measure size as the node count of the syntax tree in that normal form; without such a fixed representation "smallest formula" is not reproducible, so the grammar and size measure are part of the frame. Size and depth are minimized first; among candidates of equal size and depth, the more permissive (weaker) one is preferred, since at fixed syntactic complexity committing to less is the better explanation. We do not claim this particular lexicographic priority (size $\succ$ depth $\succ$ permissiveness) is the uniquely correct one — it is a deliberate, declared design choice, and the frame may substitute another well-founded order; what matters is that it be fixed and stated. This default has three consequences. (i) It avoids overfitting: large conjunctions such as "the entire actual trace occurred" lose on size. (ii) It avoids disjunctive drift: $A \vee B$ is syntactically larger than A, so it is chosen only when no smaller necessary cause exists — which is exactly the overdetermined case (Section 10), where the disjunction is the correct answer. (iii) The degenerate "weakest" object of Remark 3 (which excludes a single nearest counterfactual) has no small readable formula, so it never wins. To keep these guarantees we require of $\mathcal{K}_{\mathfrak{F}}$ that the trace-exclusion artifact $A \setminus \{f_0\}$ be inexpressible (which holds for the readable grammar and template families of Step 6.6); we make this an explicit condition on admissible candidate classes. Throughout we say "more permissive" for larger language and reserve "insufficient" for a candidate that does not by itself force the effect; these are kept lexically distinct.

If the necessary condition lies outside $\mathcal{K}_{\mathfrak{F}}$ — for instance $a \vee b$ when $\mathcal{K}_{\mathfrak{F}}$ admits only literals — then no valid cause exists in the class and the method reports $\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}}) = \emptyset$; the remedy is to enlarge $\mathcal{K}_{\mathfrak{F}}$, not to return a spurious singleton.

Three extremal objects appear in this document, and they must not be conflated. The following table fixes their roles once and for all.

Object	**What it is**	**Role**
$C^\star$ ($\lhd$-minimal)	syntactically smallest readable valid cause in $\mathcal{K}_{\mathfrak{F}}$ (permissiveness as tie-break)	the answ
$C^\flat$ (Prop. 1)	smallest-language anchored cause	a certifi
extensionally maximal	largest-language valid cause ($A \setminus \{f_0\}$)	degener

The *answer* is $C^\star$: among readable, effect-disjoint candidates, the order prefers the smallest formula size, then temporal depth, then — only as a tie-break — the more permissive (larger-language) necessary condition. The *certificate* $C^\flat$ is a different, τ-specific distance object — the maximal

failure-free neighborhood — that witnesses *why* the test passes but is generally not a readable formula; it is reported alongside $C^\star$, not in place of it. The *extensionally maximal* valid cause (drop a single nearest failure) is rejected outright as non-explanatory: it is both unreadable and, being huge, loses on size (Remark 3). There is thus no tension between the size-minimizing *answer*, the smallest-language *certificate*, and the excluded largest-language artifact: they are three distinct objects with three distinct purposes.

8.1 Canonical Frames

The frame $\mathfrak{F}$ has many components, and a fair objection is that a sufficiently free choice of $\mathfrak{F}$ could make almost any property a "cause," so that the cause is injected by the analyst rather than extracted from the system. We answer this not by forbidding configuration but by fixing a small number of *canonical default frames*; results should be reported against one of these unless a deviation is explicitly justified (the honesty requirement of Appendix B).

Standard trace causality $\mathfrak{F}_{\text{trace}}$
$B_{\mathfrak{F}} = \mathsf{true}$ (or the stated environment assumptions), input-only interventions (Adm $= \mathcal{L}(M)$), the count-only order $\preceq_\tau$ of Step 6.4 by default (the profile tie-break is used only when explicitly declared), effect-disjoint candidates (Assumption 2) over the observable vocabulary $I \cup O$, and $\triangleleft$ preferring smaller formula size, then smaller temporal depth, then the more permissive necessary condition. This is the default for explaining a single observed execution; the toy example uses the optional profile refinement for a unique witness.

Standard strategy causality $\mathfrak{F}_{\text{strat}}$
as above but with strategy-structure interventions over a stated editable vertex set $V_{\mathfrak{F}}$ and the edited-vertex subset order refined by trace distance (so τ remains the unique $\preceq_\tau$-minimum; see Remark 2), and effect-disjointness under the mandatory effect cone (Assumption 3) with the strategy-level exception for declared system-guarantee invariants and internal-state observables, so that strategy-level invariants such as $G(req \to F\, grant)$ become testable via the reverse parity game (Section 12).

TempoBench-style $\mathfrak{F}_{\text{TB}}$
the finite-horizon, point-effect, input-only frame of Appendix D.

Two safeguards make these defaults principled rather than tunable to taste. First, every component except the candidate vocabulary, grammar, similarity tie-break, and effect cone is fixed once the regime is chosen, and the vocabulary is constrained by effect-disjointness, so the only remaining freedom is which observable propositions are eligible — a modeling choice that must be declared. Second, under a fixed canonical frame *including those declarations* the valid-cause set $\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$ is determined by M, τ, and $E_{\mathfrak{F}}$ alone, so the cause is a function of the system and observation given that declaration, not a free parameter. (Determinacy is thus conditional on the declared vocabulary, grammar, count-only vs. profile $\preceq_\tau$, and effect cone, which must be stated.) Frame-relativity then means results are *conditional on a declared, reproducible setting*, exactly as background conditions are held fixed in ordinary causal explanation, rather than cherry-picked per query.

9 Toy Example

Consider the response specification

$$\varphi = G(req \to F\, grant).$$

It says that whenever a request occurs, a grant must eventually occur. Suppose the Mealy machine has two states, with initial state $s_0 = idle$:

$$idle \qquad \text{and} \qquad pending.$$

We fix the machine completely so that later claims about its invariants are well-defined. The transition and output functions are total:

$$\begin{array}{rcll} \delta(idle, req) & = & pending, & \lambda(idle, \cdot) = \emptyset, \\ \delta(idle, \neg req) & = & idle, & \lambda(pending, \cdot) = \{grant\}, \\ \delta(pending, \cdot) & = & idle. & \end{array}$$

In words: a request seen in *idle* moves the machine to *pending* at the next step; in *pending* the machine emits *grant* (regardless of input) and returns to *idle*; a request that arrives while in *pending* is not latched (the machine still returns to *idle*). The output depends only on the state (*grant* holds exactly in *pending*), so this toy machine is effectively a Moore machine; a Mealy machine would in general also let the output depend on the current input. Now consider an actual trace beginning with a request followed by a grant, written as valuations over $I \cup O$:

$$\{req\}, \quad \{grant\}, \quad \emptyset, \quad \emptyset, \quad \ldots$$

So the machine is in *idle* at time 0, enters *pending* at time 1 (where *grant* holds), and returns to *idle* at time 2. Let the effect be

$$E = F\, grant.$$

The minimal trace-level cause is *req***.** Take the default input-only frame, so admissible counterfactuals are exactly machine behaviors, $\mathrm{Adm}^{\mathfrak{F}}_M(\tau) = \mathcal{L}(M)$, and use the canonical count-first lexicographic closeness order of Step 6.4. Consider the candidate *req* (the proposition *req* holds at time 0). It holds on τ, and the effect $F\, grant$ holds on τ. The closest removal is the behavior with the smallest change count: it sets $\neg req$ at time 0 and changes nothing else that it need not. Since the machine starts in *idle* and now receives no request at time 0, it stays in *idle*; with the remaining inputs unchanged (all $\emptyset$) it never enters *pending* and never emits *grant*. Its difference profile is $\langle (0, \emptyset), (1, \emptyset) \rangle$, so $d = 2$ (time 1 loses the *grant* the original trace had), and $F\, grant$ fails on it. We check that *no* admissible $\neg req$-removal of count ≤ 2 preserves the effect: to emit *grant* the machine must enter *pending*, which requires a *req* in *idle* at some time $j \geq 1$; the cheapest such trace sets $\neg req$ at 0, *req* at 1, and thereby also changes the output at time 2, giving profile $\langle (0, \emptyset), (1, \{req\}), (2, \{grant\}) \rangle$ with $d = 3 > 2$. Hence every count-minimal $\neg req$-removal fails $F\, grant$, and since $\preceq_\tau$ is total the unique closest removal is the $d = 2$ behavior above. (Under the first-difference/Baire order the later-request behavior would be *tied* with the minimal removal, and under the pure change-set *inclusion* order it would be incomparable to it; in both cases it would enter Close and *req* would spuriously fail. The count-first order excludes it; see Step 6.4.) Hence *req* is non-vacuous and counterfactually necessary: by the definition of Section 8, *req is* a valid cause. Because the preference order minimizes syntactic size (and *req*, a single atom, is the

smallest readable effect-disjoint candidate that holds on τ), the candidate req is preferred to the larger $req \wedge X\, grant$. Therefore, under this frame, the formally correct minimal trace-level cause is

$$C_\tau = req,$$

not $req \wedge X\, grant$. The stronger candidate $req \wedge X\, grant$ is in fact excluded from $\mathcal{K}_{\mathfrak{F}}$ before the order is even consulted: it entails $F\, grant$ by logic alone ($\mathcal{L}(req \wedge X\, grant) \subseteq \mathcal{L}(F\, grant)$, since $grant$ at time 1 already gives $F\, grant$), so it fails clause (ii) of Assumption 2; its conjunct $X\, grant$ is a restatement of part of the effect. By contrast req does *not* entail $F\, grant$ by logic alone (only through M), so it is admitted. Even were $req \wedge X\, grant$ admitted, it would be dispreferred to req: it is syntactically larger, and on the size-minimizing order of Section 8 a single atom beats a four-symbol conjunction.

Why req can feel uninformative, and what to do about it. The intuition that "a request alone does not explain *why* the machine granted" is real, but it is a critique of the input-only *frame*, not of the candidate req. An input-only frame fixes the machine's transition and output rules as unchallengeable background facts; relative to those facts, the only thing that had to be true on the trace for the grant to appear is that a request arrived, so req is exactly the right answer. The machine's response rule does not appear in the cause precisely because the frame has placed it in the background.

To make that response rule itself a candidate explanation, one must move to a frame that re-opens it. At the strategy level, the explanatory target is a rule such as

$$C_M = G(req \rightarrow F\, grant),$$

or, if the internal state $pending$ is exposed for explanation,

$$G\big((at\_idle \wedge req) \rightarrow X\, pending\big) \wedge G(pending \rightarrow grant).$$

We must guard the first conjunct by at_idle: a request arriving in $pending$ is not latched on this machine (Section 9), so the unguarded $G(req \rightarrow X\, pending)$ is *not* an invariant of M, whereas the guarded form is. Note also that $grant$ holds in the same step as $pending$ (the machine is Moore-style), so the second conjunct is $G(pending \rightarrow grant)$, not $G(pending \rightarrow X\, grant)$. Together the two invariants entail $G\big((at\_idle \wedge req) \rightarrow X\, grant\big)$; hence on every trace whose requests arrive only in $idle$ — which includes the observed trace and its input-only counterfactuals — they imply $G(req \rightarrow F\, grant)$, the response specification. By Proposition 2, such a C_M is an invariant of M and can be tested only under a strategy-structure frame: under input-only counterfactuals it is vacuous and so fails non-vacuity. Note that C_M names the cone atom $grant$; it is admissible only because $\mathfrak{F}_{\text{strat}}$ declares it a system-guarantee invariant (Assumption 3), not at the trace level. Given that declaration, C_M also does not entail $F\, grant$ by pure logic (a request-free trace satisfies C_M vacuously while failing $F\, grant$), so clause (ii) of Assumption 2 holds as well.

The reverse game thus distinguishes these targets by changing the frame. Asking about the observed trace under the input-only frame yields the trace-level cause req. Asking which strategic rule makes the controller reliably satisfy the response effect requires a strategy-structure frame and yields the global response property. The two answers are not competitors at one level; they are the correct outputs of two different frames.

10 Overdetermination, Preemption, and the Role of the Background

Because validity is pure counterfactual necessity (Section 8), the method inherits the well-known limitations of necessity under *overdetermination* and *preemption*, and a reviewer familiar with Halpern–Pearl actual causation will expect these to be addressed head-on. We do so here, and explain why the necessity reading is a deliberate design choice rather than an oversight.

Overdetermination. Suppose two inputs a and b each, on their own, force the effect $E = X\,grant$: the machine grants at time 1 whenever a holds at time 0, and also whenever b holds at time 0, and the actual trace has both. Then neither a nor b is counterfactually necessary: removing a alone leaves b, which still forces the grant, so the closest $\neg a$-removal preserves E, and symmetrically for b. Consequently neither singleton is a valid cause. This is not a defect of the construction but the correct report under necessity: *no single input was necessary.* What *is* valid is the disjunction's failure set, captured by the cause

$$C = a \vee b,$$

whose closest removal sets *both* to false and does destroy E. The method therefore returns the disjunctive necessary condition $a \vee b$ — *provided* $\mathcal{K}_{\mathfrak{F}}$ *admits disjunctions.* If the candidate class is restricted (e.g. to single literals, as in the TempoBench-style frame), then $a \vee b \notin \mathcal{K}_{\mathfrak{F}}$, no singleton is valid, and the method correctly reports $\mathsf{Causes} = \emptyset$ (Section 8); the remedy is to enlarge $\mathcal{K}_{\mathfrak{F}}$ to include the disjunction, not to return an arbitrary singleton. In this symmetric example the certificate $C^\flat$ of Proposition 1 coincides with $a \vee b$ on the admissible set, but we caution that this identification is special to the example: in general $C^\flat$ is the distance-defined failure-free neighborhood and need not equal any particular readable disjunction. We state this behavior explicitly so that overdetermined effects are reported as disjunctive causes (when expressible), never as an arbitrary choice between symmetric inputs. This does not conflict with the readability-first preference order (Section 8), which penalizes disjunctions on size: $a \vee b$ is dispreferred to any smaller valid singleton, so it is returned *only* when no smaller valid cause exists — exactly the overdetermined case, where there genuinely is none. (We return a *necessary condition*, not "the actual cause"; necessity does not by itself adjudicate which disjunct is the actual cause.)

Preemption. If a forces E and b would have forced E only had a been absent, then again a alone is not necessary on a trace where both are present; the necessary condition is once more $a \vee b$. Distinguishing the preempting cause a from the preempted backup b requires a sufficiency or contrastive notion, which necessity does not provide. We do not claim to make this distinction; a frame that wants it must import a Halpern–Pearl-style contingency (holding b fixed at its actual value while varying a), which can be expressed as a background constraint $B_{\mathfrak{F}}$ that pins b, recovering a as the necessary cause *relative to that frame.*

Why necessity, and why this is not vacuous. Two objections must be separated. First, "necessity flags trivial conditions such as 'the system was powered on'." It does not, because of two safeguards already in the definition: non-vacuity (Section 8) discards any condition that no admissible counterfactual can violate, and the background $B_{\mathfrak{F}}$ absorbs standing operating assumptions into the setting rather than the cause (the lamp/switch discussion of Appendix B). A universally-true operating assumption is either folded into $B_{\mathfrak{F}}$ or fails non-vacuity, so it is never returned. Second, "why not sufficiency, as in Halpern–Pearl?" Halpern–Pearl actual causation

adds a minimal-sufficient-set requirement with an explicit quantifier over *contingencies* (settings of the other variables): one must exhibit a witness set whose fixing, under some contingency, both preserves the effect actually and removes it counterfactually. The complexity of deciding HP actual causation is, even in the Boolean setting, sensitive to the exact definition: it is NP-complete for the original HP definition in binary (Boolean) models, Σ_2^P-complete for the original definition in general models, and D_2^P-complete for the updated definition (Eiter–Lukasiewicz; Aleksandrowicz–Chockler–Halpern–Ivrii). Lifting any of these to ω-regular effects layers the contingency quantifier on top of language operations, so HP-style sufficiency is at least as expensive as — and conceptually heavier than — our necessity test (a one-player shortest-edit plus non-emptiness check, or, at the strategy level, a parity solve followed by a non-emptiness check; Section 13). We make only the modest claim that necessity is *no harder* (and uniform and decidable), not the stronger and unproved claim that sufficiency is *strictly* harder over ω-regular effects; a precise separation would require fixing an HP-over-ω-regular semantics, which we do not develop here. We therefore target necessity by design, report overdetermined effects as disjunctive causes, and expose contingency through $B_{\mathfrak{F}}$ when a sharper, sufficiency-like answer is wanted. This keeps the core notion uniform and decidable while leaving a clearly marked door to contrastive refinements.

11 Why the Parity Condition Matters

Many temporal properties cannot be explained by a single finite event. Examples include:

$$GF\, a, \qquad FG\, stable, \qquad G(req \to F\, ack).$$

These formulas refer to recurrence, eventual stability, and repeated response obligations. A parity automaton is designed to reason about exactly this kind of infinite behavior.

In the product $M \times A_E$, the cause may correspond to preserving an accepting recurrence pattern. For example, the controller may win because every request eventually drives the product into a strongly connected component whose recurring priority is accepting. Alternatively, the cause may be an invariant that prevents the play from entering a losing dominion after a request becomes pending.

Crucially, the parity machinery is genuinely needed only in the *strategy-structure* regime. In the input-only and output-intervention regimes the system has no free move, so (as Step 6.1 explains) the counterfactual test is a one-player shortest-edit search plus non-emptiness check, and no two-player game is solved. The remainder of this section makes the two-player object precise for the strategy-structure regime, where the title's "reverse parity game" is the operative construction.

12 The Reverse Parity Game

This section gives the two-player object that the input-only and output-intervention regimes specialize away. ("Reverse" names the *explanatory* direction — running the synthesis machinery backwards, from a frozen controller to the conditions it needed — not a retrograde or backward-time analysis; the game itself is a standard forward parity game on a partial-commitment arena.) Throughout, fix the synthesis game

$$G_\varphi = (V, V_0, V_1, T, \Omega)$$

of Section 3, the winning strategy σ implemented by M, a strategy-level candidate cause C with deterministic parity monitor $A_{\neg C} = (Q_{\neg C}, q^0_{\neg C}, 2^{I \cup O}, \delta_{\neg C}, \Omega_{\neg C})$ for its negation, and the effect

monitor A_E. The frame designates an *editable vertex set*

$$V_{\mathfrak{F}} \subseteq V_0$$

of system vertices whose moves may be re-opened; this is the same $V_{\mathfrak{F}}$ that defines the strategy neighborhood $\Sigma_{\mathfrak{F}}$ in Step 6.3 ($\sigma' \in \Sigma_{\mathfrak{F}}$ iff σ' agrees with σ on $V_0 \setminus V_{\mathfrak{F}}$).

12.1 Two Distinct Necessity Notions

Before defining the game we separate two notions that the earlier trace-based definition and the game would otherwise conflate. Fix the strategy-structure admissible set $\mathrm{Adm}_M^{\mathfrak{F}}(\tau) = \{\mathrm{Play}(\sigma', \rho) \mid \sigma' \in \Sigma_{\mathfrak{F}},\ \rho\} \cap \mathcal{L}(B_{\mathfrak{F}})$.

Trace-existential necessity (N1)
: the literal instance of Causes (Section 8): the *closest* admissible trace satisfying $\neg C$ must fail E. Here the re-opened system and the environment together pick a *single* counterfactual trace, so deciding N1 is an *existential* (one-player) shortest-edit search plus non-emptiness check over the admissible set, exactly as in Step 6.5; it is not a genuine two-player game.

Strategy-guarantee necessity (N2)
: the genuinely two-player, strategy-level question: can the system *drop the commitment* C and still *guarantee* E against every *admissible* environment? "Admissible" means satisfying the background $B_{\mathfrak{F}}$ (e.g. a fairness assumption); quantifying over *all* environments, including $B_{\mathfrak{F}}$-violating ones, would let an unfair environment trivially falsify any liveness commitment, so both clauses are relativized to $B_{\mathfrak{F}}$. Assuming the frozen σ secures the guarantee and satisfies C (the strategy-level actuality requirement), C is *strategy-necessary* iff there is **no** $\sigma' \in \Sigma_{\mathfrak{F}}$ that simultaneously (i) secures E on admissible plays, $\forall\rho.\ \mathrm{Play}(\sigma', \rho) \models (B_{\mathfrak{F}} \to E)$, and (ii) is not C-committed on an admissible play, $\exists\rho.\ \mathrm{Play}(\sigma', \rho) \models B_{\mathfrak{F}} \wedge \neg C$. (When $B_{\mathfrak{F}} = \mathsf{true}$ these are $\forall\rho.\, E$ and $\exists\rho.\, \neg C$.) N2 is centered on the actual *strategy* σ, not on τ (cf. Remark 2).

N1 and N2 are different questions — token-level (this trace) versus type-level (this controller's guarantee), in the philosophy-of-causation terminology — and we give them separate treatments. The reverse parity game decides N2; N1 is decided by the one-player check already described. We do *not* claim a single parity game decides N1, and the strategy-structure "closest-removal" reading of Causes (with the edited-vertex subset order) is an existential optimization layered on top of the one-player check, not the game below.

12.2 The Arena, Players, and Winning Condition

Definition 2 (Reverse parity game)**.** *The reverse parity game*

$$G^{\mathrm{rev}}_{M,\mathfrak{F},C,E} = (V^r, V_0^r, V_1^r, T^r, \Omega^r)$$

has vertices $V^r = V \times Q_B \times Q_E \times Q_{\neg C}$*, each pairing a game vertex with the states of deterministic monitors* A_B *(for the background* $B_{\mathfrak{F}}$*),* A_E*, and* $A_{\neg C}$*. The vertex set is* partitioned *into the* $\exists$*-player's vertices* V_0^r *and the* $\forall$*-player's vertices* V_1^r*:*

- V_0^r *(*$\exists$*, the re-opened system): vertices whose* V*-component lies in the editable set* $V_{\mathfrak{F}}$*, where* $\exists$ *may take any* G_φ*-move;*

- V_1^r *(∀): all other vertices, namely those whose V-component lies in V_1 (environment input choices)* and *those whose V-component lies in the frozen set $V_0 \setminus V_{\mathfrak{F}}$. Frozen vertices are* single-successor *(their only outgoing edge is σ's prescribed move), so assigning them to ∀ keeps the tuple a genuine partition while leaving play unaffected — their owner has no choice.*

We assume the synthesis game carries an explicit labeling $\ell : T \to 2^{I \cup O}$ *of each G_φ-edge by the input/output valuation produced on that move (the standard labeling of the synthesis game, where a round consists of an environment input choice followed by the system output); this is the label the monitors read. Edges T^r are then induced by T on the V-component together with the synchronous updates of each monitor on $\ell(e)$ along each G_φ-edge e; the edge relation is total. The ∃-player's* objective *is the assume-guarantee condition $B_{\mathfrak{F}} \to E$: ∃ wins a play π iff $\pi|_{I \cup O} \models B_{\mathfrak{F}} \to E$. This is ω-regular, so Ω^r is a parity index for it and $G^{\mathrm{rev}}_{M,\mathfrak{F},C,E}$ is a genuine parity game; A_B and $A_{\neg C}$ are also read off as* auxiliary *acceptance conditions used in the witness search below.*

12.3 Soundness

The decision procedure must be stated with care, because N2 is a *conjunctive* strategy-existence question — $\exists \sigma'$ that both secures $B_{\mathfrak{F}} \to E$ (a $\forall \rho$ condition) and admits an admissible $\neg C$ play (an $\exists \rho.\ B_{\mathfrak{F}} \wedge \neg C$ condition). **Caution:** for a parity/Büchi objective, staying inside the winning region forever does *not* imply winning (a play may remain in $W_\exists$ yet violate the parity condition), so we do *not* use a "trap" multi-strategy of "all moves staying in $W_\exists$"; instead we re-impose the objective explicitly in the witness search.

Proposition 3 (Reverse-game soundness for strategy-guarantee necessity)**.** *Assume the frozen $\sigma \in \Sigma_{\mathfrak{F}}$ secures $B_{\mathfrak{F}} \to E$ and satisfies C (the strategy-level actuality requirement). Let $W_\exists$ be the ∃-winning region of $G^{\mathrm{rev}}_{M,\mathfrak{F},C,E}$ for objective $B_{\mathfrak{F}} \to E$, and let $\mathcal{A}[W_\exists]$ be the sub-arena obtained by restricting G^{rev} to $W_\exists$ (keeping only edges that stay in $W_\exists$). Then C is* not *strategy-necessary (N2 fails) iff $\mathcal{A}[W_\exists]$ contains a play π with*

$$\pi|_{I \cup O} \models B_{\mathfrak{F}} \wedge E \wedge \neg C.$$

Equivalently, C is strategy-necessary iff the product $\mathcal{A}[W_\exists] \times A_B \times A_E \times A_{\neg C}$ has no accepting lasso for the conjunction $B_{\mathfrak{F}} \wedge E \wedge \neg C$ — a one-player ω-regular non-emptiness check (not mere reachability of a vertex).

Proof. Let Π_{win} be the plays consistent with *some* $\sigma' \in \Sigma_{\mathfrak{F}}$ that secures $B_{\mathfrak{F}} \to E$. We claim $\Pi_{\mathrm{win}} = \{\pi : \pi \text{ stays in } W_\exists \text{ and } \pi \models B_{\mathfrak{F}} \to E\}$. ($\subseteq$) A securing strategy keeps every play in $W_\exists$ and wins it, so its plays satisfy $B_{\mathfrak{F}} \to E$. ($\supseteq$) Given such a π, we may take $\pi = uv^\omega$ to be an accepting *lasso*: $\mathcal{A}[W_\exists]$ is finite and the winning condition ω-regular, so whenever a qualifying play exists a lasso one does. Define σ' to follow the (ultimately periodic) moves of π while the history matches π, and a fixed *finite-memory* winning strategy from $W_\exists$ after any environment deviation: every ∃-move on π goes to $W_\exists$ (π stays there), and the environment cannot escape $W_\exists$. Following a lasso needs only finite memory (the period), and finite-memory winning strategies exist for parity objectives on finite arenas, so σ' is finite-memory; hence $\sigma' \in \Sigma_{\mathfrak{F}}$ secures $B_{\mathfrak{F}} \to E$ and has π as a play. Now C is not strategy-necessary iff some σ' secures $B_{\mathfrak{F}} \to E$ (i) and admits a play with $B_{\mathfrak{F}} \wedge \neg C$ (ii), i.e. iff Π_{win} contains a play with $B_{\mathfrak{F}} \wedge \neg C$. By the claim such a play stays in $W_\exists$ and satisfies $B_{\mathfrak{F}} \to E$; with $B_{\mathfrak{F}}$ this gives E, so it satisfies $B_{\mathfrak{F}} \wedge E \wedge \neg C$ and lies in $\mathcal{A}[W_\exists]$. Conversely any $\mathcal{A}[W_\exists]$-play with $B_{\mathfrak{F}} \wedge E \wedge \neg C$ satisfies $B_{\mathfrak{F}} \to E$ and stays in $W_\exists$, hence lies in Π_{win} and witnesses (i)+(ii). The lasso/non-emptiness characterization is standard for deterministic ω-automata over a finite arena. □

This decision is sound *and* complete and uses only (a) a two-player parity solve for the winning region $W_\exists$ and (b) a one-player ω-regular non-emptiness check on $\mathcal{A}[W_\exists]$. It does *not* rely on a maximal-permissive-strategy construction, sidestepping the fact that for parity objectives the "moves staying in $W_\exists$" multi-strategy is *not* winning (a Bernet–Janin–Walukiewicz most-permissive *winning* strategy needs memory); the crucial point is that the objective $B_{\mathfrak{F}} \to E$ is re-imposed in the witness search, so non-winning plays that merely stay in $W_\exists$ are excluded. Step (ii) is a non-emptiness question for the monitor's ω-acceptance condition, *not* reachability of an accepting vertex. Minimizing the edited-vertex set $V_{\mathfrak{F}}$ (the $\subseteq$-closest deviation reading) is a *separate* optimization over subsets of V_0 layered on top of this decision; it is not encoded in a single parity game, and we do not claim it is (see Section 13 for its cost). The input-only and output-intervention regimes are the degenerate case $V_{\mathfrak{F}} = \emptyset$, where $V_0^r = \emptyset$, the game has a single (environment) player, and the whole question collapses to the one-player shortest-edit plus non-emptiness check of Step 6.5 for trace-existential necessity (N1).

12.4 From Game Commitments to a Trace Cause

Definition 2 also makes precise the "causal dominion extraction" of Step 6.6. Suppose C is strategy-necessary (no $\mathcal{A}[W_\exists]$-play satisfies $B_{\mathfrak{F}} \wedge E \wedge \neg C$, Proposition 3). The witness is the set of frozen commitments that block $\exists$: let

$$D_C \subseteq V_{\mathfrak{F}}$$

be the editable vertices at which σ's prescribed move is the one whose re-opening would otherwise let $\exists$ secure $B_{\mathfrak{F}} \to E$ via a $\neg C$-deviation (i.e. the commitments without which Proposition 3 would find an E-securing, C-violating play in $\mathcal{A}[W_\exists]$), and let

$$T_C = \{(v, \sigma(v)) \mid v \in D_C\} \subseteq T$$

be the corresponding σ-transitions. When the commitment is a *safety* rule (take σ's move at every visit to D_C), the induced trace-level cause is the ω-regular property

$$C \;\equiv\; G \bigwedge_{v \in D_C} \big(at\_v \to move_\sigma(v)\big),$$

read over traces with the relevant game positions exposed (Appendix's at_s convention): "whenever the play is at a committed position v, it takes σ's prescribed move," with negation $\neg C \equiv F \bigvee_{v \in D_C} (at\_v \wedge \neg move_\sigma(v))$. When the commitment is a *liveness* rule — as in the recurrence example below, where $C = GF\, at\_serve$ — the induced cause instead has the form $GF \bigvee_{v \in D_C} at\_v$ ("keep returning to a committed position"), with negation FG of its complement. In both cases breaking the commitment in T_C is exactly what an admissible $\neg C$-deviation does; this is the formal content of the slogan "a commitment becomes a rule such as $G((at\_idle \wedge req) \to X\, pending)$."

Minimal strategy cause. The strategy level has, so far, only a *validity* notion (N2), in contrast to the trace level's validity-plus-$\lhd$-minimality (Definition 1). We complete the symmetry: a *minimal strategy cause* is a $\subseteq$-minimal editable set $V_{\mathfrak{F}} \subseteq V_0$ such that the corresponding commitment C is strategy-necessary (Proposition 3); equivalently, a minimal causal dominion D_C. Unlike the trace-level order, this minimization is not a single parity solve: it is a search over vertex subsets with the parity decision as an oracle (Section 13). We therefore state plainly that the strategy level delivers *validity* via the reverse parity game and *minimality* only via this separate, combinatorially harder optimization, which we do not claim to solve in polynomial time.

12.5 A Worked Liveness Example

The trace-level toy example never exercises the parity condition because its effect $F\,grant$ is a *reachability* (co-safety) property — satisfied as soon as a grant appears, so its monitor needs no recurrence and the test reduces to reachability. We now give a recurrence effect that genuinely needs parity. Take a two-state machine over input req and output $grant$ with

$$\delta(serve, \cdot) = wait, \quad \delta(wait, req) = serve, \quad \delta(wait, \neg req) = wait,$$

$$\lambda(serve, \cdot) = \{grant\}, \quad \lambda(wait, \cdot) = \emptyset.$$

Thus, being Moore, the machine emits $grant$ exactly *while* in $serve$ (not the step after), and it enters $serve$ one step after a request arrives in $wait$. The initial state is $s_0 = wait$. Let the effect be the recurrence property

$$E = GF\,grant$$

("grants occur infinitely often"), with actual trace the lasso

$$\tau = \big(\{req\}, \{grant\}\big)^{\omega},$$

which alternates $wait \xrightarrow{req} serve \to wait$ and indeed grants infinitely often.

An environment-fairness assumption is required. Crucially, no controller can guarantee $GF\,grant$ against *every* environment: an environment that issues only finitely many requests traps the machine in $wait$, where $grant$ is never emitted. The well-posed strategy-level question therefore carries the standard environment-fairness background

$$B_{\mathfrak{F}} = GF\,req$$

("requests keep arriving"), and the guaranteed objective is the assume-guarantee property

$$B_{\mathfrak{F}} \to E \;=\; GF\,req \to GF\,grant,$$

which is again ω-regular and yields a parity game. Without $B_{\mathfrak{F}}$ the winning region for E is empty and every candidate is vacuously "necessary," so stating $B_{\mathfrak{F}}$ is what makes the example meaningful — exactly the overdetermination/vacuity discipline of Sections 8–10.

The effect monitor A_E for $GF\,grant$ is a two-priority parity (Büchi) automaton under the min-even convention: priority 0 on $grant$-valuations and priority 1 elsewhere, so a run is accepting iff priority 0 recurs infinitely often. (A single priority cannot encode "infinitely often"; two are needed.) The product $P = M \times A_E$ is accepting iff the play visits the $serve$-emitting region infinitely often — a genuine parity (recurrence) acceptance, not reachability: a counterfactual that grants a million times and then stops still *fails* E.

The strategy-level cause is the recurrence commitment. Expose the state and consider the candidate

$$C = GF\,at\_serve$$

— "the controller returns to the serving state infinitely often." (This is effect-disjoint by Assumption 2: C names at_serve, not $grant$, and $\mathcal{L}(GF\,at\_serve) \not\subseteq \mathcal{L}(GF\,grant)$ as a matter of pure logic, since the two coincide only through M.) Under the standard strategy frame $\mathfrak{F}_{\text{strat}}$ with $V_{\mathfrak{F}}$ the $wait$-vertices and background $B_{\mathfrak{F}} = GF\,req$, we apply Proposition 3. The witness search asks for a play

in $\mathcal{A}[W_\exists]$ satisfying $GF\,req \wedge GF\,grant \wedge \neg C$, i.e. $GF\,req \wedge GF\,grant \wedge FG\,at\_wait$. But *grant* is emitted only in *serve*, so $GF\,grant$ entails $GF\,at\_serve$, which contradicts $FG\,at\_wait$; no such play exists, so C is *strategy-necessary* (N2). (This uses the $B_{\mathfrak{F}}$-relativized N2: an unfair environment with finitely many requests would, against the frozen σ, yield a play with $FG\,at\_wait = \neg C$, but that play violates $B_{\mathfrak{F}} = GF\,req$ and is therefore not an admissible deviation witness — exactly why clause (ii) is relativized to $B_{\mathfrak{F}}$.) The extracted D_C is the set of *wait*-vertices from which σ moves to *serve* on a request — the accepting-recurrence-preserving commitment, the causal dominion of Section 12.4: removing the transitions T_C lets the environment force the play to remain in *wait*, out of the recurring accepting region.

By contrast, the *stronger* candidate "serve *every* request," $G((at\_wait \wedge req) \rightarrow X\,at\_serve)$, is *not* strategy-necessary: a deviation may skip some requests yet serve infinitely many others, securing $GF\,req \rightarrow GF\,grant$ on a $B_{\mathfrak{F}}$-fair play while violating the invariant, so it admits a $\neg C$ play and fails the test. This is the necessity discipline at work — the necessary commitment is the recurrence $GF\,at\_serve$, not the stronger "every request." And a reachability candidate such as "*grant* at time 1" is likewise not necessary: shifting the first grant still leaves infinitely many. Only a recurrence-preserving commitment passes, which is why the parity (recurrence) condition, not a reachability check, is the right acceptance here.

13 Decidability and Complexity

We record the cost of the two basic operations — checking a candidate and synthesizing a minimal cause — in each regime. Let $|M|$ be the number of machine states, $|A_E|, |A_{\neg C}|$ the monitor sizes, and d the number of priorities of the relevant parity condition.

We state these as conditional bounds, not as a fully proved complexity theorem; the hypotheses (monitor sizes, and how $\preceq_\tau$ is handled) are made explicit.

Input-only / output-intervention (one-player). Checking a candidate C has two parts (Step 6.5): non-vacuity is non-emptiness of $\mathrm{Adm}^{\mathfrak{F}}_M(\tau) \cap \mathcal{L}(\neg C)$ restricted to finite-change removals, and effect failure asks that the closest such removal preserve $\neg E$. We do *not* assume the closeness *relation* $\preceq_\tau$ is recognized by a fixed automaton (its primary key compares unbounded change counts, which is not a finite-state relation in general). Instead we use the constructive fact of Remark 1: for lasso τ and finite-state M, the closest removal is a *shortest-edit* (minimum-count) path in the finite product $M \times A_{\neg C} \times A_E$, computable by a shortest-path/BFS search over that product of size $O(|M| \cdot |A_{\neg C}| \cdot |A_E|)$; the effect-failure check then evaluates E on that witness. This is polynomial in the product (plus the parity-solving cost $|V|^{O(d)}$, quasi-polynomial, if E is a genuine parity rather than reachability/Büchi effect). When C ranges over LTL formulas the *deterministic* parity monitors used in the explicit product are worst-case *doubly* exponential in $|C|$ (LTL $\rightarrow$ deterministic parity is doubly exponential), so the explicit product route is doubly exponential in the formula size; the underlying language checks (non-emptiness and effect-failure) are themselves no harder than LTL model checking — PSPACE in the formula size — when carried out via *nondeterministic* monitors rather than explicit determinization. We do *not* claim a uniform PSPACE bound for the full minimum-count validity quantification. The certificate $C^\flat$ (Proposition 1) is ω-regular and computable from $\uparrow_{\preceq_\tau}(F)$ *when* that upward closure is itself ω-regular — which we obtain for the count-first order via the shortest-edit construction on lassos, and otherwise must posit as an explicit automaticity hypothesis on $\preceq_\tau$.

Minimal-cause search. Over a finite template family $\mathcal{K}_{\mathfrak{F}}$ (Step 6.6), a $\lhd$-minimal cause is found by checking each template, i.e. $|\mathcal{K}_{\mathfrak{F}}|$ checks. Over the open LTL-sketch grammar the search is unbounded; restricting to formulas of bounded size keeps it decidable (finitely many candidates) but exponential in the bound, as in syntax-guided synthesis.

Strategy-structure (two-player). Deciding strategy-guarantee necessity (N2) for a *fixed* $V_{\mathfrak{F}}$ is the procedure of Proposition 3: solve the parity game $G^{\mathrm{rev}}_{M,\mathfrak{F},C,E}$ (objective $B_{\mathfrak{F}} \to E$) of size $O(|V| \cdot |A_B| \cdot |A_E|)$ with $O(d)$ priorities to obtain the winning region $W_\exists$, then run a one-player ω-regular non-emptiness check for $B_{\mathfrak{F}} \wedge E \wedge \neg C$ on $\mathcal{A}[W_\exists]$ (product with $A_B, A_E, A_{\neg C}$; an accepting-lasso search). Parity solving is in UP $\cap$ co-UP and quasi-polynomial time, and the non-emptiness check is polynomial, so the decision is quasi-polynomial. (We do *not* invoke a permissive-strategy computation; the winning region plus the re-imposed objective suffices.) Finding a $\subseteq$-minimal $V_{\mathfrak{F}}$ that makes C necessary — the "minimal causal dominion" D_C — is a *separate* optimization over subsets of V_0: naively $2^{|V_0|}$ candidate sets, each tested by the quasi-polynomial decision above. We do not prove a tight class, but record an upper bound: checking that a given $V_{\mathfrak{F}}$ induces necessity is one quasi-polynomial parity decision, and checking minimality is a co-NP-style quantification over its proper subsets (each a parity decision); so the minimal-dominion decision sits within a polynomial-hierarchy level relative to a parity oracle (a Σ_2-type search), naturally encoded as SAT/SMT/MaxSAT over vertex-selection variables with the parity decision as oracle. Exact completeness is left open.

Finite-horizon TempoBench case. Here the candidate space $\mathcal{K}_{\mathfrak{F}_{\mathrm{TB}}}$ is finite but *exponential*: there are at most $2|I|(k+1)$ timestep-indexed literals, hence up to $2^{2|I|(k+1)}$ candidate conjunctions (constraint sets Γ). Checking one candidate is a finite reachability check over A_{HOA} restricted to the window $0, \ldots, k$, polynomial in $|A_{\mathrm{HOA}}|$ and k. Finding a subset-minimal valid cause is therefore a search over an exponential space; validity is not in general monotone in Γ (Appendix D), so greedy deletion is not sound without an extra monotonicity hypothesis, and the minimal cause is computed by exhaustive search, hitting-set, or CEGIS over the finite constraints. Each individual check remains polynomial.

14 Benchmark Architecture

The construction naturally suggests benchmark tasks for temporal causal reasoning:

1. choose an LTL specification φ;
2. synthesize or hand-design a Mealy machine M;
3. choose an actual lasso trace τ;
4. choose an effect E, usually a focused subproperty of φ;
5. compute or manually design the intended minimal cause C;
6. ask a solver to identify C from plausible distractors.

A task could ask: given this Mealy machine and the effect $F\,grant$, which temporal fact is the minimal cause? Candidate answers might include

$$req, \qquad F\,req, \qquad req \wedge X\,grant, \qquad G(req \to F\,grant), \qquad GF\,grant.$$

The correct answer depends on whether the task asks for a trace-level cause or a strategy-level cause. Making that distinction explicit is part of the point of the benchmark.

15 Main Pitfall

The phrase "minimal cause within the underlying LTL specification" can mean several different things:

1. a minimal syntactic subformula of φ;
2. a minimal trace property that explains E on τ;
3. a minimal strategy invariant of M that explains realization of φ.

These are not equivalent. For instance, if

$$\varphi = G(req \rightarrow F\, grant) \wedge G(cancel \rightarrow X\, \neg grant)$$

and the observed effect is $F\, grant$, then the relevant cause may depend on the absence of *cancel*, the current machine state, and the recurrence structure of the product automaton. It may not be a literal subformula of φ.

For this reason, the cause should not be restricted to syntactic subformulas of the original specification. A better approach is to let C be any ω-regular property in the chosen candidate class, then translate or approximate it as readable LTL when needed.

16 Certificate View

A useful output is not only a formula C, but a certificate

$$(C, W, \Pi).$$

Here C is the temporal cause and W is the accepting or winning region that the cause preserves: in the one-player regimes W is the set of product states of $P = M \times A_E$ from which the effect is still enforced (the accepting region witnessing $\tau \models E$); in the strategy-structure regime W is the $\exists$-winning region of G^{rev} (Section 12). We define Π precisely as a finite set of *counterfactual witnesses*, one per rejected weaker (or otherwise competing) candidate C': each element of Π is a lasso trace

$$\pi_{C'} = u_{C'}\, v_{C'}^{\omega} \;\in\; \text{Close}^{\mathfrak{F}}_{\tau,M}(\neg C') \cap \mathcal{L}(E_{\mathfrak{F}}),$$

that is, a closest admissible $\neg C'$-removal that *preserves* the effect, which is exactly the counterexample showing $C' \notin \mathsf{Causes}$. Such a lasso exists whenever C' fails the test and is extracted, in the one-player regimes, from the accepting cycle found by the non-emptiness check of Step 6.5, and in the strategy-structure regime from the $\exists$-winning play of Proposition 3. Thus Π is a finite, checkable record of why no preferred competitor is a valid cause.

This certificate separates three ideas:

- what happened on the observed execution,
- what had to keep happening over time,
- what would break if the proposed cause were removed.

The concise form of the construction, stated precisely, is the following.

Arena. Form the frozen product $P = M \times A_{E_{\mathfrak{F}}}$, treating M as a fixed strategy. For input-only and output-intervention frames this is the one-player arena used throughout; for a strategy-structure frame the arena is instead the original game G_φ with σ partially re-opened (Step 6.6).

Counterfactual test. *Trace-existential necessity* (N1, the core Causes definition): C passes when the closest admissible $\neg C$-traces are non-empty and all fail $E_{\mathfrak{F}}$ (Section 8); the adversary searches for an admissible nearest $\neg C$-trace that preserves $E_{\mathfrak{F}}$, and finding none is a pass. This is a one-player shortest-edit search plus non-emptiness check in *all* regimes (the system has no free move once M is frozen). *Strategy-guarantee necessity* (N2, strategy-level): the genuinely two-player reverse parity game of Section 12 asks whether the re-opened system can drop C and still force E against every environment. The two-player parity machinery is the object of N2, not of N1.

Selection. A minimal cause is any $\lhd$-minimal valid cause; when $\lhd$ is partial, several incomparable minimal causes may exist (Definition 1). Separately, the nearest-failure-anchored cause $C^\flat$ (Proposition 1) is a canonical *certificate* — the maximal failure-free neighborhood of τ — not generally a readable $\lhd$-minimal formula (Remark 3).

17 Limitations and Conclusion

What is established. We have given a frame-relative, counterfactual notion of temporal cause for synthesized controllers, decided in the common regimes by shortest-edit search plus automaton non-emptiness (Steps 6.5–6.6) and in the strategy-structure regime by a two-player reverse parity game (Section 12). Proposition 1 characterizes the nearest-failure certificate $C^\flat$; Proposition 3 relates the parity game to strategy-guarantee necessity; and Proposition 4 shows the finite construction is the literal specialization of the general definition, aligning it with TempoBench TCE.

What is conjectural or scoped. Several claims are deliberately scoped. The complexity statements of Section 13 are stated at the level of construction sizes and the standard solving bounds, not as a fully proved complexity theorem with hypotheses; they rely on the automaticity of $\preceq_\tau$, which we establish for the canonical order on lasso counterfactuals (Remark 1) but assume in the general ω-regular case. The reverse parity game decides *strategy-guarantee* necessity (N2), which is a distinct, type-level notion from the trace-existential necessity (N1) of the core definition (Section 12.1); we do not claim a single game decides N1, nor that a single game also performs the edited-vertex minimization. The relationship to Halpern–Pearl is a contrast, not a proved separation (Section 10).

Limitations. The method reports frame-relative *necessary conditions*, not full actual causes: it returns disjunctions for overdetermined effects, does not distinguish preempting from preempted causes, and returns no cause when the necessary condition lies outside the declared $\mathcal{K}_{\mathfrak{F}}$. Its outputs are determinate only after the eligible vocabulary is declared; vocabulary choice can still influence the answer (Section 8.1). The evidence here is two small worked examples plus the TempoBench alignment, now supplemented at the trace level: a full end-to-end TempoBench TCE instance exists (the publicly available sample, a six-state synthesized arbiter, whose computed $\Gamma^\star$ matches TempoBench's per-cell ground truth exactly), alongside machine-checked reproductions of five worked

trace-level instances from HOA encodings. What remains before broader empirical claims is a corpus-scale comparison (pending the dataset's re-release) and the strategy-level case study — ideally extracting a strategy-level cause from a realistic synthesized arbiter. The trace-level checker is additionally profiled (sub-second full-key computation on small windows; the actual-causation object is the measured exponential mode), and on a 107-instance random-transducer corpus the per-cell actual-cause and determining-set unions coincided on every instance.

Conclusion. The core idea — frame-relative counterfactual necessity for temporal effects over a frozen synthesized strategy, with a parity-game generalization for strategy-level causes — is, we believe, a useful organizing frame for temporal causal explanation. The present document establishes the definitions, the two regimes, and their soundness relationships; closing the remaining gaps (a proved complexity theorem, the edited-vertex minimization procedure, and an empirical evaluation) would turn this framework into a complete method.

A Symbol and Operation Dictionary

This appendix lists the mathematical objects used in the document. When a symbol has several common conventions in the literature, the convention used in this document is stated explicitly.

Logical and Temporal Notation

LTL	Linear Temporal Logic. It is a logic for writing properties of infinite traces.
$\varphi, \varphi_{\text{LTL}}$	The original LTL formula or specification. Its form is a formula over $I \cup O$, so $\varphi \in \text{LTL}(I \cup O)$, with semantics $\mathcal{L}(\varphi) \subseteq (2^{I \cup O})^\omega$. It is not the reverse-causal frame.
$\mathfrak{F}$	The reverse-causal frame. It records the scoping information used by the explanation procedure, including the selected effect, background constraints, relevant outputs, candidate-cause space, allowed interventions, similarity relation, and minimality order. This is an explanatory scope object and is unrelated to a Kripke frame in modal logic; the term "frame" is used here in the causal-context sense.
$\mathfrak{F}_\varphi$	A frame induced by φ and additional design choices η. This notation means $\mathfrak{F}_\varphi = \text{Frame}(\varphi; \eta)$, not that φ itself is the frame.
η	Additional design or scoping choices used to construct a frame, such as target selection, allowed counterfactuals, cause grammar, similarity metric, and minimality order.
$\mathfrak{F}_{\text{scope}}$	A scope-setting causal frame. It tells the reverse procedure which part of a larger effect should be treated as the target, which parts should be treated as background, and which parts may be ignored.
$\mathfrak{F}_{\text{TB}}$	A TempoBench-compatible causal frame. It restricts the effect to a positive output occurrence at one fixed timestep.
$\text{AP}(\varphi)$	The set of atomic propositions mentioned by φ.
AP	Atomic proposition. In TempoBench prompts, the AP list is the finite list of Boolean symbols used by the automaton, including both inputs and outputs.

E	The effect to be explained: a *single* (possibly conjunctive) ω-regular trace property. Its form is usually an LTL formula, a bounded temporal formula, or an equivalent language of traces. When convenient, this document uses E both for the formula and for the set of traces satisfying it; $\mathcal{L}(E)$ is the explicit language notation. The *space* of candidate effects one might choose is a separate object, written $\mathcal{E}$; $E \in \mathcal{E}$.
$\mathcal{E}$	The space (class) of candidate effects from which a particular effect E is drawn. It is a set of trace properties, not a single effect; the TempoBench-compatible effect class $\mathcal{E}_{TB}(\mathfrak{F}_{\mathrm{TB}})$ is one such space.
$E_1, \ldots, E_m$	Components of a larger effect E. For example, if $E = E_1 \wedge E_2 \wedge E_3$, then each E_r is one conjunctive obligation or sub-effect.
$E_{\mathfrak{F}}$	The part of E selected by $\mathfrak{F}$ as the target effect. If no selection is made, then $E_{\mathfrak{F}} = E$.
$B_{\mathfrak{F}}$	Background constraints supplied by $\mathfrak{F}$. Counterfactual traces may be required to satisfy $B_{\mathfrak{F}}$, so those constraints are treated as the setting of the causal question rather than as part of the cause.
$S_{\mathfrak{F}}$	The set of selected effect indices. If $E = E_1 \wedge \cdots \wedge E_m$, then $S_{\mathfrak{F}} \subseteq \{1, \ldots, m\}$ identifies which E_r's form the scoped target $E_{\mathfrak{F}}$.
$O_{\mathfrak{F}}$	The output propositions selected as relevant by $\mathfrak{F}$. Its form is a subset $O_{\mathfrak{F}} \subseteq O$.
A_{env}	Environment assumptions. These are constraints on admissible environment behavior, usually written as an LTL formula.
G_{sys}	System guarantees. These are obligations the system is intended to satisfy, usually written as an LTL formula.
C	A candidate temporal cause. Its form is usually an LTL formula, a parity or Büchi automaton, or any representation of an ω-regular trace property.
$\mathcal{K}_{\mathfrak{F}}$	The candidate-cause space allowed by $\mathfrak{F}$. Its elements are possible explanations before they have been checked for validity or minimality.
$\mathcal{I}_{\mathfrak{F}}$	The allowed intervention model supplied by $\mathfrak{F}$. It determines which counterfactual changes may be made to inputs, outputs, transitions, priorities, or strategy commitments. Note the deliberate typographic distinction from the input set I: $\mathcal{I}_{\mathfrak{F}}$ (calligraphic, frame-subscripted) is the intervention model, whereas I (upright) is the set of input atomic propositions; they are unrelated objects.
$\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$	The set of all valid causes of selected effect $E_{\mathfrak{F}}$, relative to Mealy machine M, causal frame $\mathfrak{F}$, and actual trace τ. Membership in this set means that actuality and counterfactual dependence hold.
C_{valid}	An arbitrary valid cause. Its form is an element of $\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$.
$C^{\star}$	A minimal cause (one of possibly several incomparable ones). Its form is a valid cause selected from $\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$ such that no preferred valid cause is available under the ordering $\lhd$.

C_τ — A trace-level cause for the actual trace τ. It explains one observed execution.

C_M — A strategy-level cause for the Mealy machine M. It explains a rule or invariant of the controller across its executions.

$p, a, b, g, r, req, grant, ack, cancel, pending, stable, safe, ready$ — Atomic propositions used in examples. Each is a Boolean-valued proposition at each time step. The names $g, r, req, grant, ack, cancel, pending, stable, safe, ready$ are mnemonic examples, not globally fixed symbols. Symbols such as $in_0, in_1, \ldots$ are example input proposition names.

$\neg C$ — Negation. The formula $\neg C$ holds exactly on traces where C does not hold.

$C_1 \wedge C_2$ — Conjunction. It holds on traces where both C_1 and C_2 hold.

$C_1 \vee C_2$ — Disjunction. It holds on traces where at least one of C_1 or C_2 holds. This operation is not central in the construction but is part of ordinary LTL.

$C_1 \rightarrow C_2$ — Implication. It is false only when C_1 holds and C_2 does not hold.

XC — The next-time temporal operation. It holds at time t when C holds at time $t+1$.

$X^k C$ — The next-time operation repeated k times. It holds at the current time when C holds exactly k steps later. For example, $X^3 grant$ means that $grant$ holds three steps after the current position.

FC — The eventually temporal operation. It holds at time t when C holds at some time $t' \geq t$.

GC — The always temporal operation. It holds at time t when C holds at every time $t' \geq t$.

$C_1 \, U \, C_2$ — The until temporal operation. It holds when C_2 eventually holds and C_1 holds at every earlier time until then.

$GF\,C$ — Infinitely often C. This abbreviates $G(FC)$. It means that from every point in time, C occurs again later.

$FG\,C$ — Eventually always C. This abbreviates $F(GC)$. It means that from some future point onward, C always holds.

$\models$ — Satisfaction. The expression $\tau \models C$ means that trace τ satisfies the temporal property C.

ℓ — An input literal. Its form is either i or $\neg i$, where $i \in I$ is one input atomic proposition.

c_j — A conjunction of input literals assigned to timestep j. For example, c_j could be $in_1 \wedge \neg in_3$.

C_E — A finite-horizon cause for effect E. In the TempoBench-compatible setting below, it has the form

$$C_E = \bigwedge_{j=0}^{k} X^j c_j.$$

Sets, Traces, and Languages

I	The set of input atomic propositions. Each element is controlled by the environment.
O	The set of output atomic propositions. Each element is controlled by the system.
I/O **partition**	The division of atomic propositions into input propositions I and output propositions O. It determines which player controls each proposition.
$I \cup O$	The set of all visible atomic propositions.
2^I	The powerset of I. Its elements are sets of input propositions that are true at a single time step.
2^O	The powerset of O. Its elements are sets of output propositions that are true at a single time step.
$2^{I \cup O}$	The alphabet of complete input/output valuations. Each letter is a set of propositions true at one time step.
i_t	The input valuation at time t. Its form is a set $i_t \in 2^I$.
o_t	The output valuation at time t. Its form is a set $o_t \in 2^O$.
a_t	The complete valuation at time t, defined by $a_t = i_t \cup o_t$. Its form is a set $a_t \in 2^{I \cup O}$.
α_t	A complete valuation at time t in a finite observed trace. It has the same form as a_t: a set $\alpha_t \subseteq I \cup O$.
t	A time index. Its form is a natural number $t \in \mathbb{N}$.
j, k, n, m, r	Indices. The index k is usually the timestep at which the target output effect occurs, j ranges over earlier or equal timesteps, n is the last index of a finite trace, m is the number of components in a decomposed effect, and r indexes one selected component E_r.
$\mathbb{N}$	The set of natural numbers.
τ	The actual trace being explained. Its form is an infinite word $a_0 a_1 a_2 \cdots$ over alphabet $2^{I \cup O}$.
$\tau_{\leq n}$	A finite observed trace prefix. Its form is $\alpha_0 \alpha_1 \cdots \alpha_n$, where each $\alpha_t \subseteq I \cup O$.
$\tau_{\leq k}$	The finite prefix of $\tau_{\leq n}$ ending at timestep k, also written $\tau_{\leq n}[0..k]$. It has the form $\alpha_0 \alpha_1 \cdots \alpha_k$.
$\lvert\tau_{\leq n}\rvert$	The length of the finite trace prefix $\tau_{\leq n}$. If $\tau_{\leq n} = \alpha_0 \cdots \alpha_n$, then $\lvert\tau_{\leq n}\rvert = n + 1$.
α_k	The valuation at timestep k of the finite observed trace $\tau_{\leq n} = \alpha_0 \alpha_1 \cdots \alpha_n$, using ordinary sequence indexing. We write α_k (equivalently the bracket form $\tau_{\leq n}[k]$) rather than the heavier $(\tau_{\leq n})_k$.
τ', τ''	Counterfactual or comparison traces. Their form is also an infinite word over $2^{I \cup O}$.

uv^ω A lasso representation of an infinite trace. Here u and v are finite words, and v^ω means that v repeats forever.

ω The first infinite ordinal. In this document it is used informally to denote infinite repetition or infinite-word behavior.

ω-regular property
A property of infinite traces recognizable by automata such as Büchi, Rabin, Streett, or parity automata.

$\mathcal{L}(M)$ The language generated by Mealy machine M. Its form is a set of infinite traces over $2^{I \cup O}$.

$\mathcal{L}(E)$ The language of traces satisfying effect E. Its form is a set of infinite traces.

$\mathcal{L}(C)$ The language of traces satisfying candidate cause C. Its form is a set of infinite traces.

$\emptyset$ The empty set. In an example trace, it means that no listed proposition is true at that time step.

$\subseteq$ Subset relation. $A \subseteq B$ means every element of A is also an element of B.

$\subsetneq$ Strict subset relation. $A \subsetneq B$ means $A \subseteq B$ and $A \neq B$.

$\cap$ Set intersection. $A \cap B$ contains exactly the elements that are in both A and B.

$\cup$ Set union. $A \cup B$ contains the elements that are in A, in B, or in both.

$\overline{X}$ Complement of a set or language X, relative to the ambient universe of traces under discussion.

Mealy Machines

M A Mealy machine. Its form is the tuple $M = (S, s_0, I, O, \delta, \lambda)$.

S The finite set of Mealy-machine states.

s_0 The initial Mealy-machine state. Its form is an element $s_0 \in S$.

s_t The Mealy-machine state at time t. Its form is an element $s_t \in S$.

δ The Mealy-machine transition function. Its form is

$$\delta : S \times 2^I \to S.$$

It maps the current state and current input valuation to the next state.

λ The Mealy-machine output function. Its form is

$$\lambda : S \times 2^I \to 2^O.$$

It maps the current state and current input valuation to the current output valuation.

$idle, pending$ Example state names. In the toy example, they are elements of S.

$state = s$ A notation for saying that the Mealy machine is currently in internal state s. This is not automatically visible in ordinary LTL over $I \cup O$ unless the state is exposed as an atomic proposition.

at_s An atomic proposition used to expose the internal state s to an LTL formula. For example, at_s5 means that the machine is in state s_5. We use the prefix $at\_$ rather than $in\_$ to avoid collision with input proposition names such as $in_0, in_1, \ldots$.

Automata and Parity Games

A_φ An automaton recognizing the traces that satisfy specification φ.

A_E A deterministic parity automaton for effect E. Its form is the tuple

$$A_E = (Q, q_0, 2^{I \cup O}, \delta_E, \Omega).$$

A_{HOA} The automaton supplied in HOA format in a TempoBench-style instance. It is an *acceptor* over complete valuations $2^{I \cup O}$ (inputs and outputs together), whose language is taken to be the set of admissible system behaviors; it thus plays the role of $\mathcal{L}(M)$, not of the effect monitor A_E, and is not a Mealy transducer. See Remark 4 for the faithfulness assumption.

HOA Hanoi Omega-Automata format. In this document it means the textual automaton representation used by TempoBench prompts.

Q The finite set of automaton states.

q_0 The initial automaton state. Its form is an element $q_0 \in Q$.

q_t The automaton state at time t. Its form is an element $q_t \in Q$.

$q_t \xrightarrow{\alpha_t} q_{t+1}$ A transition from state q_t to state q_{t+1} under valuation α_t.

δ_E The transition function of the automaton for E. Its form is

$$\delta_E : Q \times 2^{I \cup O} \to Q.$$

It updates the automaton state after reading one complete valuation.

Ω The priority function of a parity automaton or parity game. In this document its form is a map from states or vertices to natural numbers. The symbol Ω is reused for the game G, the effect automaton A_E, and the synthesis game G_φ; in all of them the *same* convention is in force — a run/play is winning iff the least priority seen infinitely often is even (Sections 3 and 6.1).

G_φ The parity game built from specification φ. It encodes the environment choices, system choices, and automaton progress needed to decide whether the system can force φ.

σ A system strategy. It maps the information available to the system to an output choice or game move. In finite-memory form it can be represented by a Mealy machine.

ρ	An environment behavior. Its form is an infinite sequence of environment choices, usually input valuations over I.
$\mathrm{Play}(\sigma, \rho)$	The infinite play induced by system strategy σ and environment behavior ρ. It is the trace or game path that results when the system follows σ while the environment follows ρ.
$P = M \times A_E$	The synchronous product of the Mealy machine and the automaton for E. Its states have the form $(s, q) \in S \times Q$.
(s, q)	A product state, consisting of a Mealy-machine state s and an automaton state q.
V	A set of vertices in a parity game.
T	A set of transitions or edges in a game graph.
D_C	A causal dominion or causal region associated with cause C. Its form is a set of game vertices, $D_C \subseteq V$.
T_C	A set of causal strategy commitments. Its form is a set of transitions, $T_C \subseteq T$.
W	A winning or accepting region preserved by a cause. Its form depends on the representation, but usually it is a subset of product states or game vertices.
$\mathrm{Acc}_{\mathrm{parity}}$	The parity acceptance condition. It is the property that the infinite play's recurring priority pattern is winning under the parity convention chosen for the game or automaton.
SCC	A strongly connected component of a directed graph. It is a set of vertices where every vertex in the set can reach every other vertex in the set. In parity reasoning, an accepting SCC is a recurring region whose infinite behavior satisfies the parity acceptance condition.

Counterfactual and Minimality Notation

$\preceq_\tau$	A similarity order relative to actual trace τ. The expression $\tau' \preceq_\tau \tau''$ means that τ' is at least as close to τ as τ'' is. The canonical instance used in this document is the count-first (minimal-intervention) *lexicographic total order* of Step 6.4, which is antisymmetric with τ as unique minimum; the first-difference/Baire order and the pure change-set inclusion order are coarser alternatives not used as primary.
$\prec_\tau$	The strict part of $\preceq_\tau$. It means strictly closer to τ.
$d_I(\tau, \tau')$	An input-distance measure between traces. In the finite-prefix case used as an example, it counts the number of positions where the input valuations of τ and τ' differ.
$\mathrm{Close}_{\tau,M}(\neg C)$	Shorthand for the input-only special case of $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}$, obtained by taking $B_{\mathfrak{F}} = \mathsf{true}$ and the input-only intervention model, so that $\mathrm{Adm}^{\mathfrak{F}}_M(\tau) = \mathcal{L}(M)$. It is the set of $\preceq_\tau$-minimal elements of $\{\tau' \in \mathcal{L}(M) \mid \tau' \models \neg C\}$. The frame-relative form below is the primary definition.

$\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C)$ The set of closest admissible counterfactual traces allowed by $\mathfrak{F}$ that violate C. Formally, it is the set of $\preceq_\tau$-minimal elements of

$$\{\tau' \in \mathrm{Adm}^{\mathfrak{F}}_M(\tau) \mid \tau' \models \neg C\}.$$

$\mathrm{Adm}^{\mathfrak{F}}_M(\tau)$ The admissible counterfactual traces under the frame. In the ordinary input-only case,

$$\mathrm{Adm}^{\mathfrak{F}}_M(\tau) = \mathcal{L}(M) \cap \mathcal{L}(B_{\mathfrak{F}}).$$

$\min_{\preceq_\tau} X$ The set of minimal elements of X under preorder $\preceq_\tau$. An element $x \in X$ is minimal when no element of X is strictly closer according to the preorder.

$\uparrow_{\preceq_\tau}(X)$ The upward closure of set X under preorder $\preceq_\tau$: $\{\tau'' \mid \exists\, \tau' \in X.\ \tau' \preceq_\tau \tau''\}$. Because $\tau' \preceq_\tau \tau''$ means τ' is *closer* to the actual trace τ, "above" here means *further away from* τ, not closer or better. Thus $\uparrow_{\preceq_\tau}(\neg E)$ is the set of traces that are further from τ than some effect-violating trace.

$\lhd$ A strict preference order on candidate causes. $C' \lhd C$ means that C' is preferred to C. The canonical instance (Section 8) is lexicographic: smaller formula size, then smaller temporal depth, then more permissive (larger $\mathcal{L}(C)$).

Π A finite set of counterfactual witnesses (Section 16). For each rejected competing candidate C', Π contains a lasso trace in $\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C') \cap \mathcal{L}(E_{\mathfrak{F}})$ — a closest admissible $\neg C'$-removal that preserves the effect — witnessing that C' is not a valid cause.

(C, W, Π) A certificate of explanation. C is the cause, W is the winning or accepting region it preserves, and Π records counterfactual evidence supporting minimality or rejecting alternatives.

TempoBench-Compatible Notation

TTE Temporal trace evaluation. This TempoBench task asks whether a finite trace is accepted by a supplied automaton.

TCE Temporal causal evaluation. This TempoBench task asks for the minimal timestep-indexed input constraints needed for a selected output effect.

TempoBench-style TCE
Temporal causal evaluation in the restricted benchmark sense used in Appendix C. The input is a finite-state machine, a finite trace generated by it, and a target output selected by $\mathfrak{F}_{\mathrm{TB}}$ that occurs at a particular timestep. The output is a set of timestep-indexed input conditions needed for that target output.

`XXX` g TempoBench prompt notation for $X^3 g$. More generally, k copies of `X` before an output name denote X^k applied to that output.

AP-level score
A TempoBench causality score that gives partial credit for correctly predicted atomic propositions at a timestep.

TS-level score

A TempoBench causality score that gives credit for a timestep only when the full predicted set of constraints at that timestep matches the ground truth.

$\mathcal{E}_{TB}(\mathfrak{F}_{\mathrm{TB}})$ The TempoBench-compatible effect class after $\mathfrak{F}_{\mathrm{TB}}$ has selected the relevant outputs. In this document it is

$$\mathcal{E}_{TB}(\mathfrak{F}_{\mathrm{TB}}) = \{\, X^k o \mid o \in O_{\mathfrak{F}},\ 0 \le k < |\tau_{\le n}|,\ o \in \alpha_k \,\}.$$

Its elements are positive selected-output occurrences at fixed timesteps.

o A single output atomic proposition, with $o \in O$. This differs from o_t, which is the whole set of outputs true at time t. Symbols such as o_1 and o_2 are example output proposition names.

$X^k o$ A point output effect. It means that output proposition o is true exactly k steps after the start of the finite trace.

Γ_E A set representation of a finite-horizon cause for effect E. Its elements are pairs (t, ℓ), where t is a timestep and ℓ is an input literal required at that timestep.

$\Gamma_E^\star$ The minimal finite-horizon cause set for effect E. Its elements are the timestep-indexed input literals selected as necessary.

(t, ℓ) A timestep-indexed input constraint. It means that input literal ℓ must hold at timestep t.

Common Operations in the Construction

Compilation to automata

The operation that converts an LTL formula, such as E or φ, into an automaton recognizing exactly the traces that satisfy it.

Synchronous product

The operation $M \times A_E$ that runs the Mealy machine and the automaton in lockstep on the same input/output valuation at each time step.

Freezing the strategy

The operation of treating the Mealy machine M as fixed, so the system no longer chooses outputs freely during explanation.

Counterfactual intervention

An allowed change to inputs, outputs, transitions, priorities, or strategy commitments, depending on the chosen counterfactual model.

Cause checking

The operation that tests whether

$$\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) \neq \emptyset \quad \text{and} \quad \mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) \cap \mathcal{L}(E) = \emptyset.$$

The first condition is non-vacuity (a closest removal exists); when it holds and the intersection is empty, the closest removals of C do not preserve the effect.

Cause synthesis
: The operation of searching for a valid cause C, rather than merely checking a proposed one.

Semantic minimality
: A minimality criterion based on the trace languages denoted by candidate causes, rather than only the syntactic size of formulas.

Formula-size minimality
: A minimality criterion based on the number of symbols or syntax-tree nodes in a candidate LTL formula.

Automaton-size minimality
: A minimality criterion based on the number of states, transitions, or priorities in an automaton representing the candidate cause.

Temporal-depth minimality
: A minimality criterion based on the nesting depth of temporal operators such as X, F, G, and U.

B The Original Specification φ and the Reverse Causal Frame $\mathfrak{F}$

In ordinary synthesis, φ is an LTL formula over $I \cup O$:

$$\varphi \in \mathrm{LTL}(I \cup O), \qquad \mathcal{L}(\varphi) \subseteq (2^{I \cup O})^{\omega}.$$

The reverse-causal setting does not change the type of φ. Instead, it introduces a new object, the causal frame $\mathfrak{F}$. The frame may be induced by an original specification plus additional choices:

$$\mathfrak{F} = \mathrm{Frame}(\varphi; \eta).$$

If no original φ is available, $\mathfrak{F}$ can be specified directly.

The Mealy machine M tells us what behavior is produced: for each state and input valuation, it tells us which output valuation appears next. By itself, however, the machine does not say which behavior is intended, which assumptions define the intended environment, which outputs matter, or which facts should count as explanatory rather than accidental. The role of $\mathfrak{F}$ is to provide that missing reverse-causal frame. A useful slogan is:

$$M \text{ gives behavior;} \qquad \mathfrak{F} \text{ gives explanatory scope.}$$

Thus the reverse-causal answer should be read as:

$$\text{cause of } E \text{ among behaviors of } M \text{ relative to causal frame } \mathfrak{F}.$$

If no original φ is supplied, the answer can still be frame-relative: the frame is then chosen directly rather than induced from a specification.

B.1 What $\mathfrak{F}$ Can Represent

In ordinary synthesis, φ is often the original LTL specification. In the reverse-causal setting, $\mathfrak{F}$ records the additional information needed to ask an explanatory question:

1. **Original specification.** It can state the temporal property the controller was designed to satisfy, when such a φ is available.
2. **Environment assumptions.** It can say which input behaviors are part of the intended operating environment.
3. **System guarantees.** It can say which output behaviors the system is expected to provide under those assumptions.
4. **Design intent.** It can identify which parts of the machine behavior are meaningful for the explanation.
5. **Effect scope.** It can select which part of a larger effect is the target of explanation.
6. **Vocabulary restriction.** It can restrict explanations to specification-level facts, rather than arbitrary implementation details.

This means $\mathfrak{F}$ does not have to be treated as something already fully encoded inside M, and it is not the same object as φ. It may include assumptions, relevance choices, intervention policies, or abstractions that are external to the machine. The important requirement is honesty: if a frame is used, the result must be reported as relative to $\mathfrak{F}$.

B.2 Restricting Admissible Counterfactuals

The first major use of $\mathfrak{F}$ is to restrict which counterfactual traces are allowed. Without background constraints, the reverse game may consider every trace generated by M:

$$\tau' \in \mathcal{L}(M).$$

With background constraints $B_{\mathfrak{F}}$, the reverse game can instead require

$$\tau' \in \mathcal{L}(M) \cap \mathcal{L}(B_{\mathfrak{F}}).$$

This says that counterfactual traces must still be machine behaviors, but they must also respect the intended background assumptions.

For example, suppose the machine has inputs *req* and *cancel*, and output *grant*. Suppose the machine grants exactly when a request occurs and no cancel occurs in the same step. If the actual step has *req*, no *cancel*, and *grant*, then a machine-relative cause of *grant* is

$$req \wedge \neg cancel.$$

Both parts matter if arbitrary inputs are allowed.

Now suppose the background specification includes the environment assumption

$$A_{\text{env}} = G(req \rightarrow \neg cancel).$$

This says that the intended environment never cancels at the same time as a request. If the reverse question is evaluated relative to this assumption, then the counterfactual where *req* and *cancel* happen together is not admissible. The minimal cause can become

$$req.$$

The no-cancel fact has not vanished. It has moved from the cause into the background setting.

This is the same distinction used in ordinary explanation. If a lamp turns on when someone flips a switch, we usually say the switch caused the lamp to turn on. We do not list every background condition, such as the wire being intact and the house having electricity. In this construction, $\mathfrak{F}$ tells the reverse game which facts are background conditions and which facts are candidate causes.

B.3 Selecting Which Part of E to Explain

The second major use of $\mathfrak{F}$ is effect selection. A large effect may contain several obligations:

$$E = E_1 \wedge E_2 \wedge \cdots \wedge E_m.$$

The reverse procedure may not always need to explain all of them. The scope provided by $\mathfrak{F}$ can select a subset

$$S_{\mathfrak{F}} \subseteq \{1, \ldots, m\}.$$

The selected effect is then

$$E_{\mathfrak{F}} = \bigwedge_{r \in S_{\mathfrak{F}}} E_r.$$

The reverse game computes a cause for $E_{\mathfrak{F}}$, not necessarily for all of E.

For example, suppose

$$E = G(req \rightarrow F\, grant) \wedge G(error \rightarrow F\, reset) \wedge G\neg(grant \wedge reset).$$

This combines request handling, error handling, and mutual exclusion. If the observed behavior contains a request followed by a grant, the explanation may only need the request-grant obligation. A scope-setting specification can say:

$$\mathfrak{F}_{\text{scope}} = \text{only explain the request-grant obligation.}$$

Then the selected target is

$$E_{\mathfrak{F}} = G(req \rightarrow F\, grant).$$

The reset behavior and mutual exclusion behavior need not appear in the cause unless the frame says to keep them as background constraints.

There are two common choices:

1. **Ignore the rest.** Explain $E_{\mathfrak{F}}$ and drop the unselected obligations from the reverse game.
2. **Hold the rest fixed.** Explain $E_{\mathfrak{F}}$, but require the counterfactual traces to keep satisfying the unselected background obligations.

The second choice is more like a controlled experiment: one target is varied while the rest of the contract remains stable.

B.4 Constraining the Vocabulary of Causes

The third major use of $\mathfrak{F}$ is to restrict the language in which causes are stated. A raw machine-level explanation might mention an internal state:

$$state = s_3.$$

A specification-level explanation might instead mention a temporal fact:

$$req \qquad \text{or} \qquad G(req \to F\, grant).$$

Both may be correct at different levels. The first describes implementation state. The second describes the behavior the specification made important.

This matters when the machine contains outputs or states that are irrelevant to the intended property. Suppose a controller both grants requests and toggles a heartbeat output every step. If the target effect is a grant, a purely machine-relative search may find accidental correlations involving heartbeat. If

$$\varphi = G(req \to F\, grant),$$

then the causal vocabulary can focus on req, $grant$, and related request-response facts. The heartbeat behavior can be ignored unless $\mathfrak{F}$ declares it relevant.

B.5 Avoiding Vacuous Explanations

The formula

$$G(req \to F\, grant)$$

is true on a trace with no requests. It is true because no request obligation was ever activated. This is called vacuous satisfaction. It is logically valid, but it may not be the effect one wants to explain.

If the reverse question asks why the whole formula held on a no-request trace, one possible answer is

$$G\neg req.$$

That answer is technically meaningful: no request was left unanswered because no request occurred. But if the benchmark is meant to test response reasoning, this is not the intended target.

The frame $\mathfrak{F}$ can prevent this confusion by selecting only activated obligations. If a request occurred at time 0, then the selected effect may be the particular obligation that this request eventually gets a grant. If no request occurred, the procedure can report that no non-vacuous request-grant effect was activated.

B.6 Modular Parity-Game Use

The role of $\mathfrak{F}$ also makes the parity-game construction more flexible. If a large effect decomposes as

$$E = E_1 \wedge E_2 \wedge E_3,$$

then one can build separate monitors for the components instead of treating the effect as one monolithic automaton:

$$A_E = A_{E_1} \times A_{E_2} \times A_{E_3}.$$

The frame $\mathfrak{F}$ can then select which monitor is the target, which monitors are background constraints, and which monitors are irrelevant for the current explanation.

The reverse game is therefore not only checking whether a candidate cause is necessary for a large undifferentiated property. It can ask a scoped question:

$$\text{Is } C \text{ necessary for the selected effect } E_{\mathfrak{F}} \text{ under the background } B_{\mathfrak{F}}?$$

This makes the procedure more configurable because it can explain one meaningful temporal obligation while controlling or ignoring the rest of the specification.

B.7 Restricting E to Match TempoBench

The broad reverse-parity framework allows E to be any temporal effect. To match a TempoBench-style task, $\mathfrak{F}_{\mathrm{TB}}$ restricts that broad space to a much smaller effect class.

Let $O_{\mathfrak{F}} \subseteq O$ be the output propositions selected by the TempoBench-compatible frame. Then the allowed effects are

$$E_{\mathfrak{F}} \in \mathcal{E}_{TB}(\mathfrak{F}_{\mathrm{TB}}),$$

where

$$\mathcal{E}_{TB}(\mathfrak{F}_{\mathrm{TB}}) = \{\, X^k o \mid o \in O_{\mathfrak{F}},\ 0 \le k < |\tau_{\le n}|,\ o \in \alpha_k \,\}.$$

In plain language, $\mathfrak{F}_{\mathrm{TB}}$ says:

only explain positive selected-output occurrences at fixed timesteps.

This is how $\mathfrak{F}_{\mathrm{TB}}$ restricts the broad E appendix to the TempoBench-compatible case. The unrestricted framework may explain state reachability, liveness, recurrence, or parity acceptance. The TempoBench frame selects only effects of the form

$$E_{\mathfrak{F}} = X^k o.$$

The cause is then a finite-horizon set of input constraints needed for that output at that timestep.

B.8 Why This Makes the Procedure More Configurable

Adding $\mathfrak{F}$ makes the reverse procedure applicable to a wider range of explanatory questions, for four reasons.

First, it prevents illegal or irrelevant counterfactuals from controlling the answer. Environment assumptions can be treated as background, not as causes.

Second, it lets the procedure explain the right part of a large effect. This is important when a specification contains many obligations but the question is about only one of them.

Third, it separates implementation artifacts from specification-level causes. The Mealy machine may contain states and outputs that are real but not relevant to the causal question.

Fourth, it supports benchmark design. By changing $\mathfrak{F}$, one can create a spectrum of tasks: machine-relative causes, specification-relative causes, scoped temporal causes, and TempoBench-compatible point-output causes.

The final answer should therefore always state its frame:

$$C \text{ is a cause of } E_{\mathfrak{F}} \text{ relative to } M \text{ and } \mathfrak{F}.$$

Without that statement, it is unclear whether the cause explains the whole machine behavior, the whole effect, or only the scoped effect selected by the causal frame.

C The Effect E and the TempoBench-Compatible Restriction

This appendix explains the role of E more explicitly. The main document uses E to mean "the effect we want to explain," a single trace property. The kind of property is deliberately broad: an effect can encode a state, an output, a transition, a temporal pattern, or a parity-game success condition. The space of such candidate effects — the menu one might choose E from — is denoted $\mathcal{E}$; each subsection below describes one family of members of $\mathcal{E}$. Appendix B explains how $\mathfrak{F}$ narrows a chosen E to a scoped target. When $\mathfrak{F}$ selects a target, the reverse game explains $E_{\mathfrak{F}}$, the selected part of E.

C.1 The Broad Meaning of E

The cleanest general definition is:

$$E = \text{an effect property over the generated play or trace.}$$

In language form, this can be written as

$$E \subseteq (2^{I \cup O})^{\omega}.$$

This means that E is a set of infinite traces. If the actual trace belongs to that set, then the effect occurred:

$$\tau \in E.$$

We follow the convention that $\models$ takes a formula on its right and $\in$ takes a set/language; sometimes E is written as a formula, such as $F\,grant$, in which case $\tau \models E$ and $\tau \in \mathcal{L}(E)$ coincide. When the distinction matters, the language is written explicitly as $\mathcal{L}(E)$. If a causal frame $\mathfrak{F}$ selects only part of E, then the operative effect is $E_{\mathfrak{F}}$. If no selection is made, then $E_{\mathfrak{F}} = E$.

C.2 A State Can Be an Effect

A specific reached state is a valid special case. For example, suppose the Mealy machine eventually reaches internal state s_5. We can define

$$E = F(state = s_5).$$

This means that the machine eventually reaches state s_5. The reverse causal question is then:

$$\text{What minimum temporal condition } C \text{ caused the machine to reach } s_5?$$

There is one important technical detail. Ordinary LTL specifications usually talk about visible inputs and outputs, not hidden Mealy-machine states. If the effect mentions an internal state, then either:

1. the state must be exposed as an atomic proposition, such as at_s5, or
2. the analysis must reason directly over the Mealy-machine or product-game states, where s_5 is visible to the analysis procedure.

So a state effect is legitimate, but it is not automatically an ordinary LTL property over only $I \cup O$ unless the state is made visible.

C.3 An Output Can Be an Effect

An effect can also be an output occurrence. For example,

$$E = F\,grant$$

means that the output $grant$ eventually occurs. A more precise finite-time effect is

$$E = X^k grant,$$

which means that $grant$ occurs exactly k steps after the start of the trace.

This asks:

What caused this output to appear at this time?

Possible causes might be local facts about the trace, such as

$$req \qquad \text{or} \qquad req \wedge X\, grant,$$

or a strategy-level rule such as

$$G(req \rightarrow F\, grant).$$

The right kind of answer depends on the causal frame. A trace-level question looks for what caused this output on this execution. A strategy-level question looks for what rule in the controller makes this output behavior reliable.

C.4 A Transition Can Be an Effect

The effect can be a transition rather than a state. For example,

$$E = F\big((state = s_2) \wedge X(state = s_3)\big)$$

means that the machine eventually moves from state s_2 to state s_3. This is useful when the interesting causal question is:

Why did the controller choose this branch rather than another branch?

Transition effects are richer than simple reachability effects because they focus on a decision made by the controller.

C.5 A Temporal Pattern Can Be an Effect

The most useful effects are often temporal patterns. Suppose the original specification is

$$\varphi = G(req \rightarrow F\, grant).$$

Several different effects could be chosen:

$$F\, grant,$$

meaning that a grant eventually occurred;

$$req \rightarrow F\, grant,$$

meaning that a request was answered;

$$G(req \rightarrow F\, grant),$$

meaning that all requests are always eventually answered; or

$$GF\, grant,$$

meaning that grants occur infinitely often.

These effects are all meaningful, but they ask different causal questions. If $E = F\, grant$, the cause may be local. If $E = G(req \rightarrow F\, grant)$, the cause is probably a recurring response rule. If $E = GF\, grant$, the cause may involve an infinite loop or recurrence structure.

C.6 A Parity-Game Success Condition Can Be an Effect

In the parity-game view, E need not be a single state or output. It can be the success condition of an infinite play:

$$E = \mathrm{Acc}_{\mathrm{parity}}.$$

This means that the play satisfies the parity acceptance condition. More concretely, E could mean that the play eventually remains in an accepting SCC, or that the priorities recurring infinitely often form a winning pattern. Strictly speaking $\mathrm{Acc}_{\mathrm{parity}}$ is a property of the *run* (the priority sequence of the product $P = M \times A_E$), not directly of the $I \cup O$ trace. We make the trace property well-defined by tying it to the admissible traces, on which P really does have runs. Because A_E is deterministic, each admissible trace $\tau' \in \mathcal{L}(M)$ has a *unique* run in P; define the parity-acceptance effect, relative to admissible traces, by

$$\mathcal{L}(E) = \{\, \tau' \in \mathcal{L}(M) \mid \text{the unique run of } \tau' \text{ in } P \text{ is accepting} \,\},$$

which is equally the projection onto $I \cup O$ of the accepting runs of P (every run of P sits over an M-behavior, so the projection lands inside $\mathcal{L}(M)$). This is an ω-regular trace property, so E remains the same kind of object as the bounded and liveness effects above; for traces outside $\mathcal{L}(M)$ there is no run and the effect is simply not evaluated, which is harmless since the admissible set is always $\subseteq \mathcal{L}(M)$ in the regimes that use this effect.

This matters because parity games are about infinite behavior. Reaching one state is not always enough. A play can temporarily visit a state that looks bad and still win later. It can also visit a state that looks good and later lose. The decisive fact is the infinite recurring pattern.

For parity reasoning, a natural effect is therefore:

the play reaches and remains in a region whose infinite behavior is accepting.

This is why the broad reverse-parity construction treats E as a temporal property, not merely as a reached state.

C.7 Hierarchy of Possible Effects

The candidate effects can be arranged in an informal order of *increasing richness* (this is a heuristic reading-aid, not a strict expressiveness ordering; a state effect is not in general less expressive than an output effect):

$$\begin{aligned} &\text{state effect} \rightsquigarrow \text{output effect} \rightsquigarrow \text{transition effect} \\ \rightsquigarrow\ &\text{bounded temporal effect} \rightsquigarrow \text{unbounded liveness effect} \\ &\rightsquigarrow \text{parity acceptance effect.} \end{aligned}$$

Here $\rightsquigarrow$ reads "progresses to a typically richer kind," and carries no claim of strict containment between consecutive entries.

Examples are:

state effect	$F(state = s_5)$	reachability causality,
output effect	$F\,grant$	output causality,
transition effect	$F\big((state = s_2) \wedge X(state = s_3)\big)$	decision causality,
response effect	$req \rightarrow F\,grant$	temporal dependency,
invariant effect	$G\,safe$	safety reasoning,
recurrence effect	$GF\,ready$	infinite-time reasoning,
parity effect	$\mathrm{Acc}_{\mathrm{parity}}$	game-theoretic temporal reasoning.

The most general lesson is:

E is whatever temporal fact the reverse game is being asked to explain.

A state can be that fact, but it is only one special case.

C.8 TempoBench-Style Temporal Causality

The broad definition of E is useful for reverse-parity explanation, but it is too broad for direct comparison with TempoBench-style Temporal Causality Evaluation. To make the setting TempoBench-compatible, the causal frame $\mathfrak{F}_{\mathrm{TB}}$ restricts the selected effect $E_{\mathfrak{F}}$. In the restricted TempoBench-style setting used here, the task is:

1. start with a finite-state machine;
2. observe a finite trace generated by that machine;
3. choose an output that occurs at a particular timestep;
4. identify the input facts at each timestep that were necessary for that output to occur.

Thus the selected effect should not initially be an arbitrary temporal formula. To match this setting, $E_{\mathfrak{F}}$ should be observable, output-based, finite-horizon, and timestep-indexed.

C.9 Finite Trace Notation

The body of the document uses τ for an infinite trace. For the TempoBench-compatible restriction, we use a finite observed prefix:

$$\tau_{\leq n} = \alpha_0, \alpha_1, \ldots, \alpha_n,$$

where each

$$\alpha_t \subseteq I \cup O$$

is the complete input/output valuation at timestep t.

The length of this finite trace is

$$|\tau_{\leq n}| = n + 1.$$

Using ordinary sequence indexing, the valuation at timestep k is written α_k (equivalently $\tau_{\leq n}[k]$), and the prefix ending at timestep k is $\tau_{\leq k} = \tau_{\leq n}[0..k] = \alpha_0 \cdots \alpha_k$.

For a single atomic proposition o and a valuation $\alpha_t \subseteq I \cup O$ (a set of propositions), satisfaction at that position is set membership: o holds at time t iff $o \in \alpha_t$, and the literal $\neg o$ holds iff $o \notin \alpha_t$. We use membership for atoms and reserve $\models$ for satisfaction of a temporal formula by a trace, e.g. $\tau \models F\, grant$.

Temporal operators on these finite prefixes are read with LTL_f (LTL over finite traces) semantics. Equivalently, since every effect considered here is strictly bounded by the horizon $k < |\tau_{\leq n}|$, one may evaluate the prefix under ordinary infinite-trace LTL by appending an arbitrary infinite extension; the truth value of $X^k o$ does not depend on the extension.

C.10 The TempoBench-Compatible Effect Class

The clean restricted effect class uses the outputs selected by $\mathfrak{F}_{\mathrm{TB}}$. Let

$$O_{\mathfrak{F}} \subseteq O$$

be those selected outputs. Then:

$$\mathcal{E}_{TB}(\mathfrak{F}_{\mathrm{TB}}) = \{\, X^k o \mid o \in O_{\mathfrak{F}},\ 0 \le k < |\tau_{\le n}|,\ o \in \alpha_k \,\}.$$

In plain English:

$$\boxed{E \text{ should be “this output occurred at this exact timestep.”}}$$

Here $o \in O$ is a single output atomic proposition, and k is the timestep where that output is observed. In the TempoBench-compatible case, o must also be selected by $\mathfrak{F}_{\mathrm{TB}}$, so $o \in O_{\mathfrak{F}}$. For example,

$$E_{\mathfrak{F}} = X^3 grant$$

means that *grant* occurs at timestep 3.

This is a special case of the broader effect definition. The broad definition allows arbitrary temporal facts. The TempoBench-compatible frame $\mathfrak{F}_{\mathrm{TB}}$ keeps only positive selected-output occurrences at fixed finite positions.

C.11 The Corresponding Cause Form

For a TempoBench-compatible effect

$$E_{\mathfrak{F}} = X^k o,$$

the cause is finite-horizon. It should describe input conditions at timesteps

$$0, 1, \ldots, k$$

that are necessary for output o to occur at timestep k.

One formula form is:

$$C_E = \bigwedge_{j=0}^{k} X^j c_j,$$

where each c_j is a conjunction of input literals. An input literal is either an input proposition or its negation. For example:

$$in_1, \qquad \neg in_3, \qquad in_1 \wedge \neg in_3.$$

An equivalent set form is:

$$\Gamma_E = \{(t, \ell) \mid 0 \le t \le k \text{ and } \ell \text{ is a required input literal at time } t\}.$$

For example,

$$\Gamma_E = \{(1, in_2), (3, \neg in_0)\}$$

corresponds to the bounded temporal formula

$$C_E = X\, in_2 \wedge X^3 \neg in_0.$$

The reverse-causal test asks whether these input constraints are necessary. If one of the required constraints is removed or flipped while the counterfactual trace remains as close as possible to the actual trace, then the target output should disappear. If the output still occurs, the proposed cause was not necessary.

C.12 What to Exclude for Direct TempoBench Alignment

To stay directly aligned with the TempoBench-style task, the initial effect class should exclude:

$$F\,grant,$$

$$G(req \to F\,grant),$$

$$GF\,ready,$$

$$\mathrm{Acc}_{\mathrm{parity}},$$

and

$$F(state = s_7).$$

These are all meaningful effects for the broader reverse-parity project, but they change the task. The first three are temporal effects that are not tied to one output at one finite timestep. The parity effect is an infinite-play success condition. The state effect is internal-reachability-focused unless the state is explicitly exposed.

For direct TempoBench compatibility, the restriction should be:

$$\boxed{E_{\mathfrak{F}} = X^k o \quad \text{where } o \in O_{\mathfrak{F}} \text{ and } o \in \alpha_k.}$$

It is also best to start with positive output occurrences only. That means effects such as $X^k o$, not $X^k \neg o$. Asking why an output did not occur is a non-event causality question, which is usually more ambiguous because many different facts can prevent an output.

C.13 Point-Effect and Property-Effect Causality

It is useful to name the restricted and broad versions separately.

Point-effect temporal causality. This is the TempoBench-compatible version:

$$E_{\mathfrak{F}} = X^k o.$$

The effect is one positive output at one timestep. The cause is a finite set of timestep-indexed input constraints.

Property-effect temporal causality. This is the broader reverse-parity version. The effect E may be a bounded or unbounded temporal property, such as a response property, recurrence property, state reachability property, transition property, or parity acceptance property.

A simple progression is:

1. Output at time: $X^k o$. This is TempoBench-compatible.
2. Output conjunction at time: $X^k(o_1 \wedge o_2)$. This is a small extension.
3. Bounded output pattern: $X^2 a \wedge X^5 b$. This is an extension.
4. Bounded response: $X^j req \to X^k grant$. This is an extension.
5. State reachability: $F(state = s)$. This is a different task.
6. Full LTL effect: $G(req \to F\,grant)$. This is not TempoBench-compatible.

7. Parity acceptance: $\mathrm{Acc}_{\mathrm{parity}}$. This is not TempoBench-compatible, but it is reverse-parity-specific.

Thus the broad reverse-parity idea can explain many kinds of effects, but the TempoBench-compatible frame $\mathfrak{F}_{\mathrm{TB}}$ selects the narrow class

$$\boxed{\mathcal{E}_{TB}(\mathfrak{F}_{\mathrm{TB}}) = \{\, X^k o : o \in O_{\mathfrak{F}} \,\},}$$

with k fixed by the observed finite trace and with the additional requirement that the output actually occurs:

$$o \in \alpha_k.$$

Under this restriction, the reverse-parity method becomes an alternate formal way to solve a TempoBench-style causal task: compute the smallest input constraints whose removal prevents the target output from occurring at the chosen timestep.

D TempoBench Alignment from the Reverse Question Onward

This appendix explains how the reverse-parity procedure should be changed when the goal is to match a TempoBench-style temporal causal evaluation (TCE) task. The purpose is not to replace the general construction. The purpose is to specialize it so that the solver receives the same kind of information TempoBench gives, solves the same kind of finite-horizon causal problem, and returns a *necessity*-based answer of the same shape. One caveat throughout: TempoBench labels TCE with CORP, a Halpern–Pearl *actual-causation* tool in Halpern's *modified* variant (its contingency mechanism resets outputs to their actual-trace values; verified against the primary sources in the companion verification suite). The TempoBench authors confirm the shipped keys are generated with contingencies *disabled*, so each key is the contingency-free counterfactual—the subset-minimal joint input-flip. This construction instead computes counterfactual *necessity*, which differs on overdetermined effects (Section 10: necessity returns the no-pivotal-cell verdict where the keys return the joint flip-set), so what follows is an *alternate*, necessity-based account of the TCE task, not a reproduction of CORP's answer key (the actual-cause object that matches the keys is developed in the companion TempoBench paper).

This appendix is based on the TempoBench paper and public implementation. TempoBench has two tasks: temporal trace evaluation (TTE), which asks whether a finite trace is accepted by a supplied automaton, and temporal causal evaluation (TCE), which asks for the minimal input constraints needed for a specified output at a specified timestep.[1]

D.1 What TempoBench Gives the Solver

A general reverse-parity problem may require choosing an effect, choosing an actual trace, compiling monitors, and deciding what counterfactuals are allowed. A TempoBench-style instance fixes most of this information in advance.

The solver is given:

1. an automaton A_{HOA} in HOA format;

2. the list of APs used by the automaton;

[1] TempoBench paper: https://arxiv.org/abs/2510.27544. Public implementation: https://github.com/nik-hz/tempobench.

3. a finite observed trace $\tau_{\leq n}$;

4. one or more effects to analyze, usually written in the form `XXX` g, meaning $X^3 g$;

5. enough information in the prompt or dataset to know which output is the target effect and which APs are eligible input causes.

The solver is not asked to synthesize a controller, choose a trace, choose an effect, discover the original LTL specification, or explain an infinite parity condition. It is asked to perform finite-horizon causal credit assignment over the provided execution.

D.2 Specializing the Reverse Question (Section 5)

The general reverse question is

$$M + \varphi + \mathfrak{F} + E + \tau \longrightarrow C^\star.$$

For TempoBench-style TCE, this becomes

$$A_{\mathrm{HOA}} + \mathfrak{F}_{\mathrm{TB}} + \tau_{\leq n} + E_{\mathfrak{F}} = X^k o \longrightarrow C^\star.$$

The meaning of each component is restricted:

A_{HOA}	The supplied finite-state automaton. This replaces the need to synthesize or reconstruct a controller.
$\mathfrak{F}_{\mathrm{TB}}$	The benchmark frame. It says that the only target effects are positive output occurrences at fixed finite timesteps, and that the causes should be timestep-indexed input constraints.
$\tau_{\leq n}$	The supplied finite trace. The solver does not choose it.
$E_{\mathfrak{F}} = X^k o$	The selected point effect. It says that output o occurs at timestep k.
$C^\star$	The minimal cause, represented as a finite set of timestep-indexed input literals.

So the TempoBench-compatible reverse question is:

> Which input facts in the provided prefix were necessary for output o at timestep k?

D.3 Specializing Step 6.1: Freeze the System Strategy

In the general construction, this step freezes the system strategy M and forms the product $P = M \times A_E$ with a deterministic parity automaton for the effect. In a TempoBench-style instance the correspondence is concrete, but it requires us to be precise about what kind of object A_{HOA} is.

Remark 4 (What A_{HOA} is, and the faithfulness assumption). TempoBench presents the system as a finite-state *acceptor* $A_{\mathrm{HOA}} = (Q, 2^{I \cup O}, \delta, q_0, F)$ over the alphabet of complete valuations $2^{I \cup O}$: it reads valuation sequences $\alpha_0 \alpha_1 \cdots$ (each $\alpha_t \subseteq I \cup O$, inputs *and* outputs together) and accepts a finite trace when its run from q_0 ends in an accepting state of F (this is exactly TempoBench's trace-acceptance, or TTE, semantics). It is presented as an acceptor over $I \cup O$ rather than as an input/output transducer: it does not read inputs and emit outputs. Its language is a set of valuation sequences, and we take $\mathcal{L}(A_{\mathrm{HOA}})$ to be the set of admissible system behaviors. In this sense A_{HOA}

plays the role of $\mathcal{L}(M)$ in the general construction (it supplies the admissible behaviors), *not* the role of the effect monitor A_E.

For the reduction to be faithful we make two assumptions, both checkable against the benchmark artifact. **(F1) Behavior model.** $\mathcal{L}(A_{\mathrm{HOA}})$ is exactly the set of executions of the system under analysis, so "accepted by A_{HOA}" coincides with "is a genuine behavior"; if instead it were a pure specification monitor A_φ whose language is the set of *specification-satisfying* traces, identifying it with the system model would conflate A_φ with M, and we exclude that reading. **(F2) Functionality.** Since TempoBench's automata are obtained by reactive synthesis from a controller specification, the behavior set is *functional in the inputs*: each input sequence has at most one accepted $I \cup O$ completion (the automaton is input-deterministic). This is *consistent* with the acceptor framing — an acceptor over $I \cup O$ may perfectly well recognize a functional input/output relation — and it is exactly the property the operational reading of Step 6.5 below uses to talk about "the" output at each timestep. We state (F1)–(F2) as explicit, benchmark-level assumptions rather than building them into the data type, so that the reduction's correctness is hostage to stated facts about TempoBench rather than to a silent convenience. The causal task itself (inputs as candidate causes, outputs as effects) presupposes (F2). **We have not independently verified (F1)–(F2) against the released TempoBench HOA artifacts**; whether those automata are input-deterministic system models (as opposed to specification monitors) is a checkable property of the benchmark that we leave to be confirmed against the artifacts. Accordingly the alignment of Appendix D is stated *conditionally on (F1)–(F2)*: if they hold, the reduction (Proposition 4) is exact; if the supplied automata are specification monitors, (F1) fails and the alignment does not apply.

Given Remark 4, the point effect $E_{\mathfrak{F}} = X^k o$ needs no separate parity monitor at all, since it is a bounded membership check on the trace at timestep k (namely $o \in \alpha_k$). There is therefore no product to build and no parity acceptance to track: the "semantic arena" reduces to A_{HOA} together with the bounded effect, and the solver receives A_{HOA} directly.

Therefore this step becomes:

1. parse the HOA automaton;
2. parse the AP list;
3. identify the target output o from the effect $X^k o$;
4. identify the input APs that may appear as causes;
5. parse the supplied finite trace $\tau_{\leq n}$;
6. if state information is provided, verify the finite transition sequence

$$q_t \xrightarrow{\alpha_t} q_{t+1}$$

for the relevant prefix.

The important constraint is that there is no new synthesis problem here. The automaton is treated as the fixed system model. The reverse procedure should use the same information a TempoBench solver sees: the automaton, the trace, the AP names, and the target effect.

D.4 Specializing Step 6.2: Choose the Actual Trace

In the general construction, an actual trace must be chosen. In a TempoBench-style instance, this step is already fixed by the benchmark. The solver is given a finite observed trace

$$\tau_{\leq n} = \alpha_0, \ldots, \alpha_n$$

generated by the automaton. Therefore, the solver does not choose τ. Instead, it parses the provided trace, verifies the state sequence if needed, and restricts attention to the prefix

$$\tau_{\leq k}$$

when the target effect has the form

$$E_{\mathfrak{F}} = X^k o.$$

The reason for this restriction is simple: later timesteps cannot be causes of an output at timestep k in this finite-horizon task. The relevant causal window is:

$$0, 1, \ldots, k.$$

If the trace contains a cycle marker, such as a `cycle` annotation with a loop index, that marker belongs to the trace representation but is not the target of the finite causal query. The causal query only needs the finite prefix up to the effect.

D.5 Specializing Step 6.3: Define the Allowed Counterfactuals

In the general construction, one must decide whether counterfactuals may change inputs, outputs, transitions, priorities, or strategy commitments. TempoBench uses a narrower frame. This step instantiates $\mathcal{I}_{\mathfrak{F}_{\mathrm{TB}}}$, the intervention component of the TempoBench-compatible causal frame.

For TempoBench-style TCE, use finite input-only counterfactuals:

1. the automaton A_{HOA} is fixed;
2. the target output o is not manually intervened on;
3. transitions are not changed;
4. priorities or acceptance conditions are not changed;
5. strategy-level commitments are not changed;
6. candidate causes are input literals at timesteps $0, \ldots, k$.

A cause is represented as a finite set

$$\Gamma_E = \{(t, \ell) \mid 0 \leq t \leq k \text{ and } \ell \text{ is an input literal}\}.$$

The corresponding bounded temporal formula is

$$C_E = \bigwedge_{(t,\ell)\in\Gamma_E} X^t \ell.$$

The benchmark allows positive and negative constraints. For example, a cause may require in_2 at timestep 1, or it may require $\neg in_0$ at timestep 3. If no input constraint is needed at a timestep, the output format may record that timestep as having no constraints.

D.6 Specializing Step 6.4: Define Similarity

The general construction needs a similarity relation over infinite traces. A TempoBench-style instance should not use that full infinite-trace machinery. The trace is finite, and the target effect occurs at a fixed timestep.

The closest counterfactuals should therefore be finite-prefix perturbations of

$$\tau_{\leq k}.$$

They should change only the input constraints being tested and should keep the rest of the prefix as close as possible to the provided trace.

For this setting, the most natural closeness rule is:

fewer changed timestep-literal constraints means closer,

counted in changed *input cells*—under (F2) the outputs are determined by the inputs, so only input changes are counterfactual choices, and these (not the full-valuation positions of the general order) are what we count. Minimality is then set-theoretic. A cause is preferred when it uses fewer or strictly necessary timestep-indexed literals. The benchmark does not ask for a semantically weakest ω-regular property, a smallest parity automaton, or a shortest LTL formula.

D.7 Specializing Step 6.5: Check a Candidate Cause

In the general reverse-parity game, an adversary tries to violate the candidate cause while preserving the effect. In TempoBench-style TCE, the same idea is finite and concrete.

Given a candidate Γ_E, first check actuality:

1. every listed literal $(t, \ell) \in \Gamma_E$ holds in the supplied trace at timestep t;
2. the target output holds at timestep k:

$$o \in \alpha_k.$$

Then check counterfactual dependence. The adversary tries to remove or flip a listed input constraint while keeping the rest of the relevant prefix close to the original trace. The candidate is valid only if the target effect disappears:

$$X^k o \text{ no longer holds under the allowed counterfactual.}$$

Operationally, this is a finite reachability check over A_{HOA}, not an infinite parity acceptance check. Since A_{HOA} is a finite-state acceptor over complete valuations (Remark 4), the test is phrased as the existence of an accepted valuation sequence: after imposing the perturbed input constraints, does there exist a finite valuation sequence $\alpha'_0 \cdots \alpha'_n$ such that (i) it is accepted by A_{HOA} in the sense made precise below (its run ends in F), (ii) its inputs match the perturbed input constraints, (iii) it agrees with $\tau_{\leq n}$ elsewhere as the closeness order requires, and (iv) $o \in \alpha'_k$? If such a sequence exists, the effect survives the intervention and the proposed cause is *not* necessary; if no such sequence exists, the proposed cause passes the counterfactual test. Under the functionality assumption (F2) of Remark 4 (each input sequence admits at most one accepted completion), this existence question coincides with the familiar reading "simulate the perturbed inputs and read off whether o appears at time k".

D.8 Specializing Step 6.6: Synthesize the Minimal Cause

The general construction offers several possible synthesis routes: full automata-theoretic cause synthesis, CEGIS over LTL sketches, or causal dominion extraction. For TempoBench-style TCE, the output should be much narrower.

The result should be a finite-horizon cause:

$$C^\star = \bigwedge_{(t,\ell)\in\Gamma_E^\star} X^t \ell,$$

where $\Gamma_E^\star$ is the minimum set of timestep-indexed input literals needed for $X^k o$.

TempoBench's public prompt style expects a JSON-like answer organized by effect and timestep. For example, an effect written as `XXX` g, meaning $X^3 g$, may have an answer shaped like:

$$\begin{aligned} &\text{timestep } 0: && \text{no constraints,}\\ &\text{timestep } 1: && \text{no constraints,}\\ &\text{timestep } 2: && \text{no constraints,}\\ &\text{timestep } 3: && r. \end{aligned}$$

The mathematical object behind that answer is

$$\Gamma_E^\star = \{(3, r)\}.$$

Thus, for TempoBench alignment, synthesis should not return a broad LTL explanation such as $G(req \rightarrow F\, grant)$, and it should not return a strategy-level invariant. It should return the minimal finite set of input constraints, placed at the correct timesteps.

D.9 Trace-Level, Not Strategy-Level

The trace-level versus strategy-level distinction is important. TempoBench TCE is a trace-level point-effect task. The question is not:

Why does the whole controller realize φ?

It is:

Which provided input facts caused this output at this timestep?

Therefore, the TempoBench-compatible version should use:

$$E_{\mathfrak{F}} = X^k o$$

and should return a finite cause over $\tau_{\leq k}$. It should not try to explain the whole LTL specification, a full parity-winning condition, or the controller's entire strategy.

D.10 Formal Specialization

Rather than informally "reading" the general ω-regular machinery as finite, we give the finite-horizon objects explicitly, *entirely in finite-trace terms*; no infinitary acceptance condition for A_{HOA} is needed or used. Fix the supplied automaton A_{HOA}, the finite trace $\tau_{\leq n}$, and the point effect $E_{\mathfrak{F}} = X^k o$ with $k \leq n$. The candidate space $\mathcal{K}_{\mathfrak{F}_{\mathrm{TB}}}$ is the finite set of conjunctions of timestep-indexed

input literals over the window $0, \ldots, k$, identified with their constraint sets $\Gamma \subseteq \{0, \ldots, k\} \times \mathrm{Lit}(I)$, where $\mathrm{Lit}(I) = \{i, \neg i \mid i \in I\}$:

$$C_\Gamma = \bigwedge_{(t,\ell)\in\Gamma} X^t \ell.$$

There are at most $2|I|(k+1)$ such literals, hence $|\mathcal{K}_{\mathfrak{F}_{\mathrm{TB}}}| \leq 2^{2|I|(k+1)}$ candidate constraint sets: the candidate space is finite but *exponential* in the window, not linear.

Finite-horizon admissible set. We make finite acceptance precise, since the supplied A_{HOA} is a finite-state acceptor with accepting set F (Remark 4). A finite valuation sequence $\beta = \beta_0 \cdots \beta_n$ is *accepted* iff it has a run

$$q_0 \xrightarrow{\beta_0} q_1 \xrightarrow{\beta_1} \cdots \xrightarrow{\beta_n} q_{n+1} \quad \text{with } q_{n+1} \in F,$$

i.e. all transitions are defined and the run ends in an accepting state; this is TempoBench's TTE acceptance and is the single, fixed meaning of "accepted" used throughout this appendix. (Under the functionality assumption (F2) of Remark 4, the run is unique once the inputs are fixed.) Writing $\mathcal{L}_{\leq n}(A_{\mathrm{HOA}})$ for the set of accepted length-$(n{+}1)$ sequences, the bounded admissible set anchored at $\tau_{\leq n}$ is

$$\mathrm{Adm}^{\leq k}_{A,\mathfrak{F}_{\mathrm{TB}}} = \mathcal{L}_{\leq n}(A_{\mathrm{HOA}}).$$

Because $X^k o$ depends only on positions $0, \ldots, k$, it suffices to track prefixes through timestep k; we keep the full length n only so that "accepted" refers to a complete run of the supplied trace.

Finite-horizon closeness and removals. For a candidate Γ, a removal is an admissible β that violates C_Γ (i.e. some $(t, \ell) \in \Gamma$ fails in β). The closeness order counts changed timestep-literal constraints, and the closest removals are

$$\mathrm{Close}^{\leq k}_{\tau_{\leq n},A}(\neg C_\Gamma) = \min_{\#\text{changes}} \{\beta \in \mathrm{Adm}^{\leq k}_{A,\mathfrak{F}_{\mathrm{TB}}} \mid \beta \not\models C_\Gamma\}.$$

Finite-horizon valid causes.

$$\begin{aligned}\mathsf{Causes}^{\leq n}_{A,\mathfrak{F}_{\mathrm{TB}},k}(X^k o) = \{\, \Gamma \mid\ & \tau_{\leq n} \models C_\Gamma,\ o \in \alpha_k, \\ & \mathrm{Close}^{\leq k}_{\tau_{\leq n},A}(\neg C_\Gamma) \neq \emptyset, \\ & \forall\, \beta \in \mathrm{Close}^{\leq k}_{\tau_{\leq n},A}(\neg C_\Gamma).\ o \notin \beta_k \,\}.\end{aligned}$$

A minimal cause is $C^\star = C_{\Gamma^\star_E}$, where $\Gamma^\star_E$ is any subset-minimal element of $\mathsf{Causes}^{\leq n}_{A,\mathfrak{F}_{\mathrm{TB}},k}(X^k o)$ (several incomparable ones may exist); minimality is by set inclusion over Γ, not by LTL formula size, temporal depth, or automaton size.

Remark 5 (Validity is not monotone; greedy deletion is unsound in general). Membership in $\mathsf{Causes}^{\leq n}_{A,\mathfrak{F}_{\mathrm{TB}},k}$ is not monotone under $\subseteq$ on Γ: enlarging Γ can make a candidate valid (a removal now has to flip more, and the cheapest removal may stop preserving the effect) or invalid (the closest $\neg C_\Gamma$-removal may flip an *irrelevant* added literal while keeping the target output). Hence the naive greedy procedure "delete literals one at a time while validity holds" is not guaranteed to reach a subset-minimal valid cause, and computing $\Gamma^\star_E$ requires genuine search over the exponential space — by exhaustive enumeration, a hitting-set/MaxSAT encoding, or CEGIS over the finite constraints. Each individual validity check is polynomial (finite reachability over A_{HOA} on the

window $0, \ldots, k$); it is the *search* that is hard. If one additionally posits that validity is upward- or downward-closed for a particular automaton, greedy deletion becomes sound, but we do not assume this in general.

Proposition 4 (Finite specialization — a definitional correspondence)**.** *This is a* definitional *restatement, not a deep theorem: the finite-horizon objects above were* defined *to mirror the general clauses, and the content is that the mirroring is exact and needs no infinitary acceptance condition on A_{HOA}; we record it as a proposition only to state that correspondence precisely. Let $\mathfrak{F}_{\mathrm{TB}}$ be the input-only frame with candidate space $\mathcal{K}_{\mathfrak{F}_{\mathrm{TB}}}$, background $B_{\mathfrak{F}} = \mathsf{true}$, bounded admissible set $\mathrm{Adm}^{\leq k}_{A,\mathfrak{F}_{\mathrm{TB}}} = \mathcal{L}_{\leq n}(A_{\mathrm{HOA}})$ (finite acceptance as defined above), the change-counting closeness order on the window $0, \ldots, k$, and subset minimality, and assume (F1)–(F2) of Remark 4. Interpret the general clauses of* Causes *(Section 8) over finite traces with LTL_f semantics on the window. Then for every Γ, $\Gamma \in \mathsf{Causes}^{\leq n}_{A,\mathfrak{F}_{\mathrm{TB}},k}(X^k o)$ iff C_Γ satisfies the four valid-cause clauses on $\tau_{\leq n}$ under $\mathfrak{F}_{\mathrm{TB}}$. In particular, the finite construction* is *the general definition restricted to the point effect; no claim is made about, and nothing depends on, infinite extensions of $\tau_{\leq n}$ or an ω-language of A_{HOA}.*

Proof. Each of the four clauses of Causes refers only to positions $0, \ldots, k$: actuality is $\tau_{\leq n} \models C_\Gamma$ and $o \in \alpha_k$; non-vacuity is non-emptiness of the admissible $\neg C_\Gamma$-removals; and effect-failure quantifies over the closest such removals, asking $o \notin \beta_k$. The effect $X^k o$ and every literal $X^t \ell$ ($t \leq k$) is determined by the length-$(k{+}1)$ prefix under LTL_f semantics, so each clause is a finite statement about prefixes in $\mathcal{L}_{\leq n}(A_{\mathrm{HOA}})$. These are verbatim the clauses of $\mathsf{Causes}^{\leq n}_{A,\mathfrak{F}_{\mathrm{TB}},k}$: the admissible sets coincide by definition, the closeness order is in both cases the count-only coarsening of $\preceq_\tau$ on the window (the change-counting preorder of Step 6.4, so that Close is the full set of minimum-count removals and the effect-failure clause quantifies over all of them), and (F2) makes "o at position k" a well-defined function of the admissible inputs. Hence the two memberships are literally the same predicate. No infinite object appears in any clause, which is why the reduction needs no ω-acceptance for A_{HOA}. □

D.11 Evaluation Compatibility

TempoBench evaluates causal answers at two granularities.

AP-level A prediction can receive partial credit for correctly identifying individual atomic propositions at a timestep.

TS-level A timestep receives credit only when the whole set of constraints predicted for that timestep matches the ground truth.

This means the reverse-pipeline output should preserve timestep structure. A flat formula such as

$$X\, in_2 \wedge X^3 \neg in_0$$

is mathematically fine, but a TempoBench-compatible answer should also preserve the grouped representation:

$$\begin{aligned} 1: &\quad in_2, \\ 3: &\quad \neg in_0. \end{aligned}$$

That grouping matches how AP-level and TS-level scoring are computed.

TempoBench also controls difficulty with finite, observable features: effect depth k, number of automaton states, transition count, trace length, causal input count, and number of unique inputs in the trace. The pipeline of this appendix should preserve those features. It should not add hidden information from the original LTL specification or from unrelated files, because doing so would make the adapted task easier than the benchmark task.

D.12 Checklist for TempoBench Alignment

To make the reverse-parity pipeline match TempoBench-style TCE, use the following restrictions:

1. Use the provided HOA automaton as A_{HOA}.
2. Use the provided finite trace $\tau_{\leq n}$.
3. Use the provided effect $E_{\mathfrak{F}} = X^k o$.
4. Restrict causal attention to $\tau_{\leq k}$.
5. Allow only finite input-literal counterfactuals.
6. Do not intervene directly on outputs.
7. Do not change transitions, priorities, or strategy commitments.
8. Minimize by necessary timestep-indexed input constraints.
9. Return causes grouped by timestep.
10. Treat broader effects, liveness, parity acceptance, and strategy-level explanations as extensions beyond TempoBench.

With these restrictions, the reverse-parity framework becomes a formal explanation procedure for the same kind of problem TempoBench tests: determining which earlier input facts were necessary for a later output in a provided finite automaton trace.